

Actividad 6 - Entrega final del proyecto

JUAN SEBASTIAN MARTINEZ HIGUITA

CRISTIAN DAVID RESTREPO ORTIZ

GIOVANNY ANATOLI CARO DAZA

JIMMY ALEXANDER MENJURA CALVO

FACULTAD DE INGENIERÍAS, UNIVERSIDAD IBEROAMERICANA

PROYECTO DE SOFTWARE

DTE. TATIANA CABRERA

Actividad 6 - Entrega final del proyecto

JUAN SEBASTIAN MARTINEZ HIGUITA

CRISTIAN DAVID RESTREPO ORTIZ

GIOVANNY ANATOLI CARO DAZA

JIMMY ALEXANDER MENJURA CALVO

FACULTAD DE INGENIERÍAS, UNIVERSIDAD IBEROAMERICANA

PROYECTO DE SOFTWARE

DTE. TATIANA CABRERA

24 DE MAYO DE 2025

Tabla Contenido

Introducción	1
Abstract	2
Necesidad	3
Alcance	4
Restricciones	5
Criterios de Aceptación	6
Descripción del Problema	7
Objetivos	8
Objetivo General	8.1
Objetivo Especifico	8.2
Fase de Diseño	9
1. RQF-RQNF	9.1
2. HU (enlace tablero de la metodología ágil)	9.2
Modelamiento	10
Diagrama de clases-diagrama de objetos- diagrama de componentes	10.1
Diagrama de casos de uso - diagrama de secuencias -diagrama de estados	10.2
nociones básicas de diseño: prototipos de baja fidelidad	10.3
-prototipos de alta fidelidad(enlace de herramienta utilizada) - Mapa de navegación.	10.4
Fase de Desarrollo	11
Codificar min. 2 funcionalidades según la necesidad planteada y los RQF	11.1
Repositorio enlace de GitHub	11.2
 Manual Técnico (Código)	
 Explicación por módulo	
Fragmentos relevantes de código documentado	
 Enlace al proyecto desplegado	

Pruebas Realizadas

Pruebas unitarias

Enlace video de como funciona el sistema

procesos futuros.	13
Conclusiones	14
Referencias	15

INTRODUCCIÓN

En la actualidad, la gestión eficiente de la cadena de suministro es un factor clave para el éxito de cualquier empresa que maneje mercancías a nivel nacional e internacional. Con el crecimiento del comercio global, la necesidad de contar con un sistema confiable y preciso para el monitoreo y control de los productos en tránsito se ha vuelto fundamental.

La empresa XXX, que se encarga del transporte de mercancías a diferentes partes del mundo por vía terrestre, marítima o aérea, ha venido presentando diversos inconvenientes en el proceso de transporte. Entre los problemas más comunes se encuentran los atrasos en la entrega de mercancías debido a fallas logísticas, errores en la coordinación de envíos y demoras en el despacho. Además, se han reportado daños en la mercadería transportada, lo que genera insatisfacción en los clientes y pérdidas económicas para la empresa.

Estos problemas han afectado la reputación de la compañía y han generado la necesidad de implementar una solución tecnológica que permita mejorar la gestión del transporte. Por esta razón, la empresa ha decidido desarrollar un sistema que centralice y optimice el control de los procesos de envío, seguimiento y entrega de mercancías. Este sistema permitirá a la empresa monitorear en tiempo real el estado de cada envío, prevenir retrasos mediante la optimización de rutas y mejorar la seguridad de los productos transportados.

Además, el sistema proporcionará herramientas para gestionar la comunicación con los clientes, notificándose sobre el estado de sus envíos y permitiéndoles realizar un seguimiento detallado de sus pedidos. Con estas mejoras, la empresa busca garantizar una mayor satisfacción del cliente, optimizar sus operaciones logísticas y mantener su competitividad en el mercado global del transporte de mercancías.

Este proyecto tiene como objetivo el desarrollo de un sistema de control de mercancía que permita a los dueños de los productos conocer en todo momento la ubicación exacta de sus envíos, en qué bodega se encuentran y por qué medios de transporte están siendo trasladados, ya sea por barco, avión o camión.

El sistema propuesto proporcionará información en tiempo real, mejorando la toma de decisiones y reduciendo los riesgos asociados a la pérdida, retrasos o daños en la mercadería. Adicionalmente, permitirá a los usuarios optimizar la logística de sus operaciones, minimizando costos y mejorando la eficiencia en la distribución.

Mediante el uso de tecnologías avanzadas como IoT, bases de datos seguras y geolocalización, el sistema será capaz de recopilar, procesar y mostrar información relevante para cada envío. Esto no solo permitirá un mejor seguimiento de la mercadería, sino también una mayor transparencia y confianza en la relación entre proveedores, transportistas y clientes.

Abstract

En el contexto actual del comercio global, la eficiencia en la gestión de la cadena de suministro representa un factor crítico para el éxito de las empresas dedicadas al transporte de mercancías. La empresa XXX, especializada en el traslado de productos por vía terrestre, marítima y aérea, ha enfrentado múltiples desafíos operativos, tales como retrasos en las entregas, errores de coordinación logística, y daños en la mercadería. Estas dificultades han impactado negativamente en la satisfacción del cliente y la reputación de la compañía, generando la necesidad de adoptar una solución tecnológica integral.

Este proyecto propone el desarrollo de un sistema inteligente de control y monitoreo de mercancías, diseñado para optimizar el seguimiento en tiempo real, mejorar la seguridad de los productos en tránsito y fortalecer la comunicación con los clientes. El sistema utilizará tecnologías como geolocalización (GPS), Internet de las Cosas (IoT) y bases de datos seguras para ofrecer visibilidad total sobre la ubicación, medio de transporte y almacenamiento de cada envío. Además, incluirá funcionalidades como alertas por incidentes, generación de reportes y protección de datos mediante cifrado.

Con esta iniciativa, se busca minimizar los riesgos logísticos, reducir costos operativos, mejorar la toma de decisiones y reforzar la competitividad de la empresa en el mercado internacional de transporte de mercancías.

Necesidad

Las empresas de transporte enfrentan constantemente el desafío de mantener el control sobre el envío y la recepción de mercancías. En particular, la empresa ***** dedicada al transporte marítimo, terrestre y aéreo ha experimentado problemas recurrentes relacionados con el extravío de mercancías durante el trayecto. Esto genera pérdidas económicas, afectación a la reputación y problemas de gestión.

Para abordar esta problemática, se ha decidido implementar un software especializado que permita registrar y monitorear el envío de mercancías, garantizando el control desde el punto de partida hasta la llegada a destino. Además, se pretende utilizar sistemas de geolocalización (GPS) para rastrear en tiempo real el trayecto de los vehículos, controlar el tiempo estimado de llegada, el estado de la mercancía, el vehículo que realiza el traslado y la bodega de recepción.

Alcance del Proyecto

El sistema de control de mercancía abarca los siguientes aspectos:

- Monitoreo en tiempo real: Los dueños de la mercancía podrán visualizar la ubicación de sus productos en cualquier momento.
- Registro de almacenamiento: Identificación de la bodega en la que se encuentra la mercancía y su historial de movimientos.
- Seguimiento de transporte: Información detallada sobre los medios utilizados para el traslado (barco, avión o camión).
- Seguridad de datos: Protección de la información mediante cifrado y protocolos de acceso seguro.

- Alertas y notificaciones: Envío de alertas en caso de retrasos, cambios de ruta o incidentes con la mercancía.
- Generación de reportes: Creación de informes sobre el estado y movimiento de la mercancía para mejorar la gestión logística.
- Rastreo satelital:
-

Restricciones

- El sistema dependerá de la disponibilidad de internet para la actualización en tiempo real.
- No incluye gestión de pagos o facturación de envíos.
- La precisión de la geolocalización estará sujeta a la cobertura de los dispositivos GPS utilizados.

Criterios de Aceptación del Proyecto

- Permitir la consulta en tiempo real de la ubicación de la mercancía.
- Garantizar la seguridad de la información con autenticación de usuarios.
- Proporcionar reportes detallados de los movimientos de la mercancía.
- Ser accesible desde dispositivos móviles y computadoras con conexión a internet.

Descripción del Problema

La empresa XXX, que se encarga del transporte de mercancías a diferentes partes del mundo por vía terrestre, marítima o aérea, ha venido presentando diversos inconvenientes en el proceso de transporte. Entre los problemas más comunes se encuentran los atrasos en la entrega de mercancías debido a fallas logísticas, errores en la coordinación de envíos y demoras en el despacho. Además, se han reportado daños en la mercadería transportada, lo que genera insatisfacción en los clientes y pérdidas económicas para la empresa.

Estos problemas han afectado la reputación de la compañía y han generado la necesidad de implementar una solución tecnológica que permita mejorar la gestión del transporte. Por esta razón, la empresa ha decidido desarrollar un sistema que centralice y optimice el control de los procesos de envío, seguimiento y entrega de mercancías. Este sistema permitirá a la empresa monitorear en tiempo real el estado de cada envío, prevenir retrasos mediante la optimización de rutas y mejorar la seguridad de los productos transportados.

Además, el sistema proporcionará herramientas para gestionar la comunicación con los clientes, notificándose sobre el estado de sus envíos y permitiéndoles realizar un seguimiento detallado de sus pedidos. Con estas mejoras, la empresa busca garantizar una mayor satisfacción del cliente, optimizar

sus operaciones logísticas y mantener su competitividad en el mercado global del transporte de mercancías.

Objetivos

Objetivo General: Desarrollar un software de gestión y monitoreo para la empresa *****, que permita el control eficiente de los envíos y la recepción de mercancías, optimizando la trazabilidad y reduciendo el riesgo de pérdida o daño durante el trayecto.

Objetivos Específicos:

1. Implementar un sistema de registro para el control de los envíos y las recepciones en las bodegas.
2. Integrar un módulo de geolocalización (GPS) para monitorear en tiempo real el recorrido de los vehículos.
3. Desarrollar un sistema de control que registre el tiempo de desplazamiento y el estado de la mercancía desde el punto de partida hasta el destino.
4. Permitir el almacenamiento de información sobre el vehículo utilizado y la bodega de recepción, asegurando la trazabilidad completa del proceso.

Fase de Diseño

Requisitos Funcionales y No Funcionales (RQF - RQNF)

Requisitos Funcionales (RQF)

- **RQF1:** El sistema debe permitir registrar envíos logísticos.
- **RQF2:** El sistema debe permitir consultar el estado de un envío.
- **RQF3:** El sistema debe permitir gestionar inventarios por bodega.
- **RQF4:** El administrador puede asignar rutas de entrega.

Requisitos No Funcionales (RQNF)

- **RQNF1:** La aplicación debe estar disponible 99% del tiempo.
- **RQNF2:** Las respuestas del backend no deben superar 2 segundos.
- **RQNF3:** El sistema debe ser escalable horizontalmente.
- **RQNF4:** Cumplir con OWASP para evitar vulnerabilidades comunes.

Historias de Usuario (HU)

Para el desarrollo de esta plataforma se va a usar jira como herramienta de control de versiones acontinuacion se muestra un resumen de las historias de usuario mas usadas dentro del sistema

Historias de Usuario por Módulo

Autenticación y Gestión de Usuarios

HU-001: Como administrador, quiero poder registrar nuevos usuarios en el sistema para gestionar su acceso.

HU-002: Como usuario, quiero iniciar sesión en el sistema con mi correo y contraseña para acceder a mis funciones.

HU-003: Como administrador, quiero asignar roles y permisos a los usuarios para restringir el acceso a ciertas funciones.

HU-004: Como usuario, quiero poder recuperar mi contraseña en caso de olvido para restablecer mi acceso.

Gestión de Transportistas y Vehículos

HU-005: Como administrador, quiero registrar transportistas en el sistema para asignarles envíos.

HU-006: Como operador logístico, quiero consultar la lista de transportistas disponibles para optimizar asignaciones.

HU-007: Como transportista, quiero recibir notificaciones sobre los envíos asignados para planificar mis rutas.

HU-008: Como administrador, quiero gestionar la información de los vehículos para mantener un registro actualizado.

HU-009: Como transportista quiero gestionar el envío de información al sistema con el fin de reportar novedades con el fin de solucionar algún posible retraso.

Gestión de Mercancías y Almacenes

HU-0010: Como cliente, quiero registrar un nuevo envío en el sistema para solicitar su transporte.

HU-0011: Como operador logístico, quiero asignar mercancías a transportistas para gestionar su distribución.

HU-0012: Como administrador, quiero consultar el estado de todas las mercancías para monitorear su ubicación.

HU-0013: Como cliente, quiero recibir una notificación cuando mi mercancía haya sido entregada.

Seguimiento y Rastreo de Envíos

HU-0014: Como cliente, quiero rastrear mi envío en tiempo real para conocer su ubicación actual.

HU-0015: Como operador logístico, quiero recibir alertas de retrasos para tomar decisiones correctivas.

HU-0016: Como transportista, quiero actualizar el estado del envío en el sistema para mantener informados a los clientes.

HU-0017: Como cliente, quiero recibir notificaciones de cambios en el estado de mi envío para estar al tanto.

Módulo de Solicitudes

HU-0018: Como cliente, quiero enviar solicitudes de transporte de mercancías para gestionar mis

envíos.

HU-019: Como operador logístico, quiero revisar y aprobar solicitudes de transporte para garantizar disponibilidad.

HU-020: Como cliente, quiero cancelar una solicitud antes de que sea confirmada para evitar costos innecesarios.

HU-021: Como transportista, quiero recibir notificaciones sobre nuevas solicitudes asignadas para planificar mis entregas

Notificaciones y Reportes

HU-022: Como administrador, quiero generar reportes sobre envíos completados, retrasos y problemas para analizar el rendimiento.

HU-023: Como cliente, quiero recibir un correo o mensaje cuando mi mercancía sea entregada para confirmar la recepción.

HU-024: Como operador logístico, quiero recibir alertas de posibles problemas en la entrega para tomar medidas.

HU-025: Como transportista, quiero enviar alertas sobre accidentes o problemas en ruta para notificar retrasos.

Modelamiento

Diagrama de arquitectura del sistema

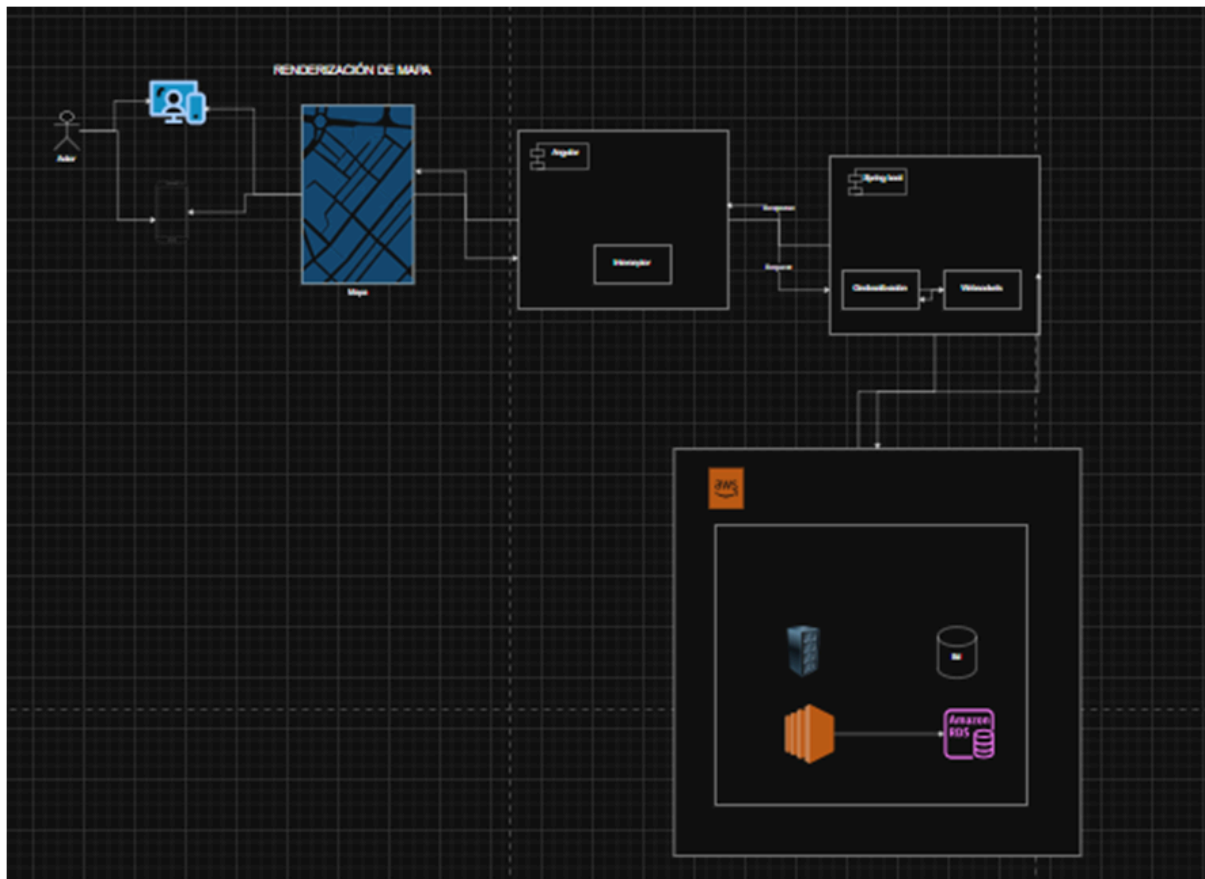


Diagrama de flujo

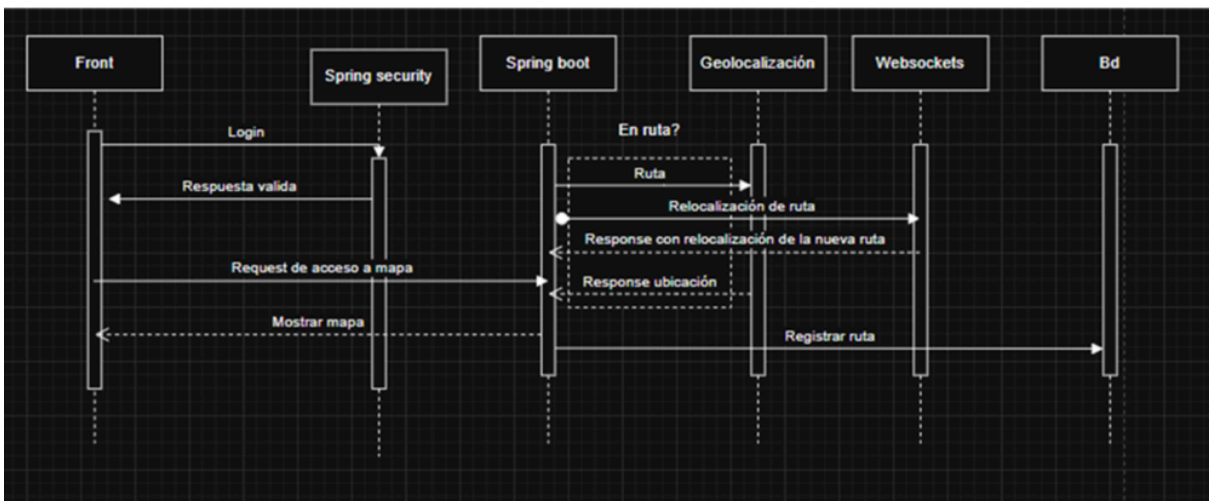
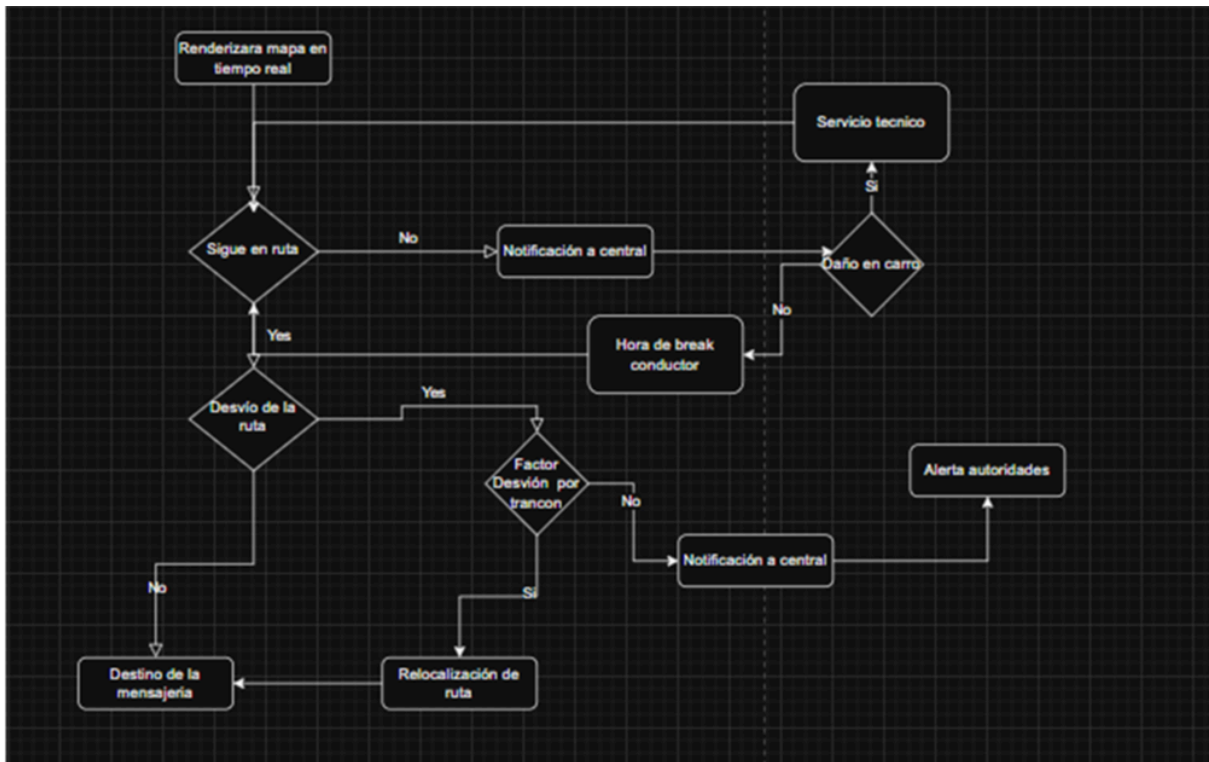
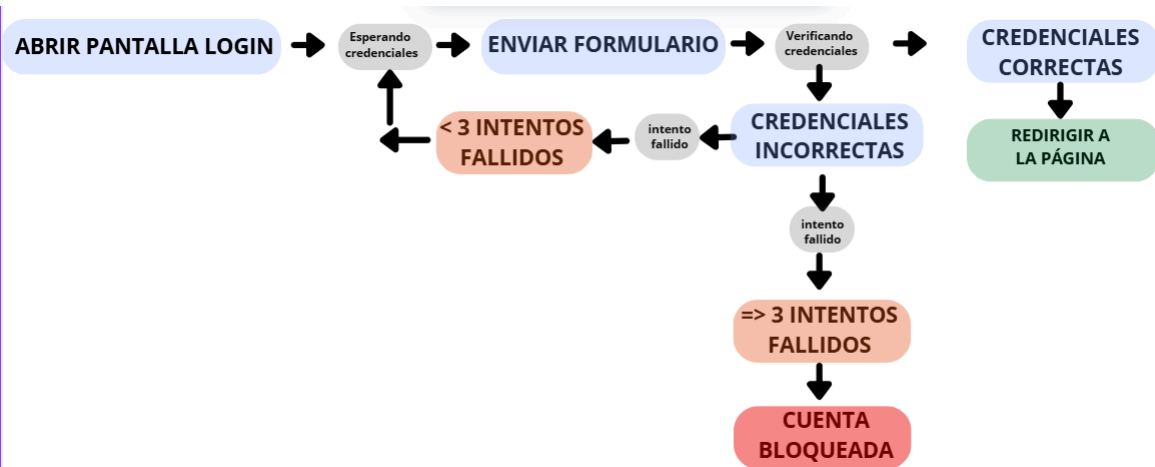
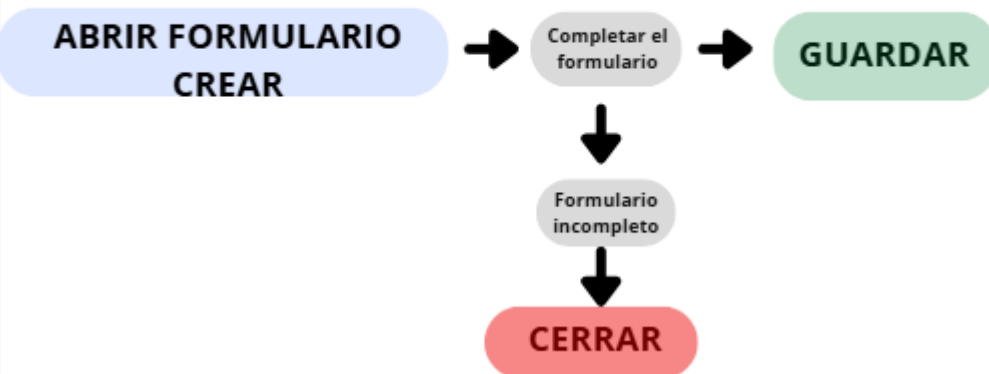


DIAGRAMA DE ESTADOS DEL LOGIN



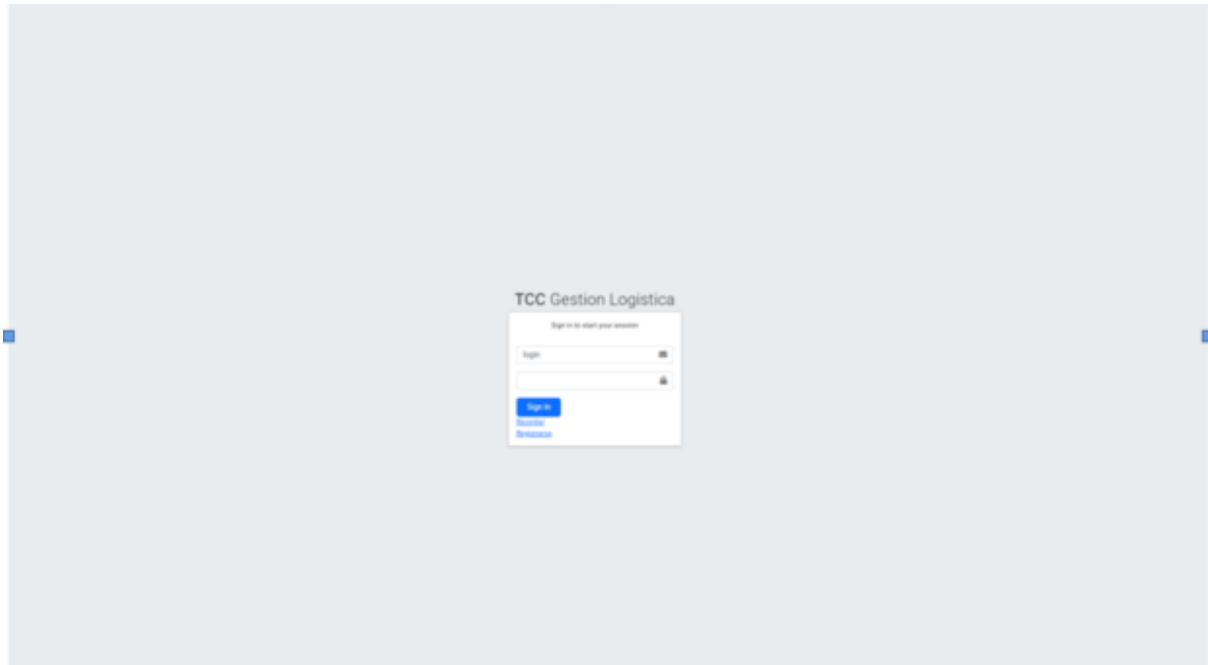
CREAR ENVÍO

CREAR ENVÍO

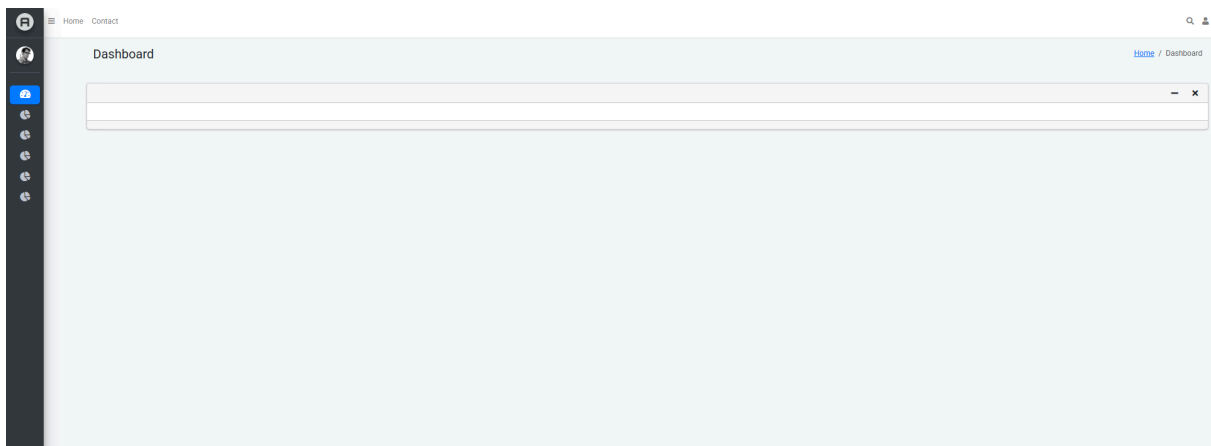


ESTADOS DE ENVÍOS

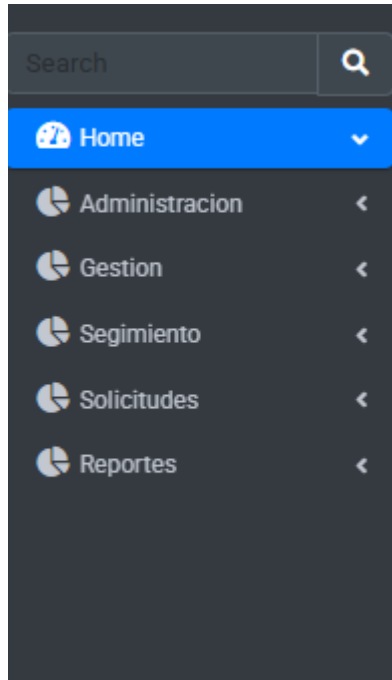




Dashboard Prototipo

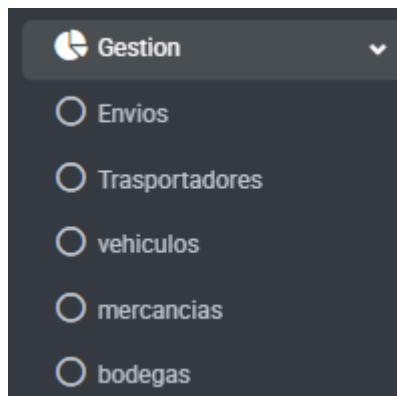


Menu



- **RQF2:** El sistema debe permitir consultar el estado de un envio.

Menu Envios



seccion envios

ID	Codigo Cliente	Telefono	Codigo Bodega	Fecha Envio	Fecha Entrega	Ciudad Origen	Ciudad Destino	Estado	Accion
101	2	3160405672	2	12/02/2025	15/03/2025	1	1	1	Editar Eliminar
101	2	3160405672	2	12/02/2025	15/03/2025	1	1	1	Editar Eliminar
12	2	3160405672	2	12/02/2025	15/03/2025	1	1	2	Editar Eliminar
104	2	3160405672	1	12/02/2025	15/03/2025	1	1	1	Editar Eliminar

nuevo envio RQF1: El sistema debe permitir registrar envios logísticos.

REFERENCIA

Cliente

Telefono

Bodega

Fecha Entrega

Ciudad Destino

Fecha Envio

Ciudad Origen

Estado

Cargar Mercancias

Mercancias

Cantidad

Descuento

Estado

+ Agregar Mercancia

Id	Producto	Cantidad	Precio	Total	Accion
----	----------	----------	--------	-------	--------

Cancelar

Guardar

lo mismo en editar envío

Para Acceder a esta sección en el menú seleccionar el módulo gestión y a continuación damos en el enlace a envíos

Caso Uso 1 :Permitir la autenticación de los usuarios.

El sistema debe permitir el ingreso del nombre y password del usuario para realizar las diferentes funciones que tendrá cada uno.

ENTRADAS: Cédula, Nombres, Apellidos, Contraseña, Tipo de Usuario (Administrador, Operador, Transportista,cliente).

PROCESOS El sistema pedirá la correspondiente identificación como administrador,. Nos ubicamos

en la parte de Administración del menú principal y escogemos Usuario El sistema pedirá los correspondientes datos del nuevo usuario luego verificará que no haya espacios en blanco, en el caso de ningún error guardará los datos del nuevo usuario.

SALIDAS Las salidas van dirigidas a el Superadministrador o Administrador Del sistema . Mensaje de error en el caso de no haber llenado algún campo. Mensaje de error en el caso de ingresar un numero de cedula ya existente en la base de datos Mensaje de error en casos de ingresar incorrectamente los datos es decir que el formato de los datos sea incorrecto

Caso de Uso 1: Registrar Envío

OBJETIVO: Permitir que el sistema registre un nuevo envío ingresado por un cliente, operador o administrador.

ENTRADAS:

documento del cliente

teléfono de contacto del cliente

bodega de envío de la mercancía

fecha de entrega

fecha envio

ciudad de origen del envío

ciudad destino del envío

estado del envío

la referencia (descripción Tipo de envío normal, express, etc.)

PROCESOS:

Desde el menú principal de la opción gestión, el usuario selecciona la opción "Envío". El sistema presenta un formulario modal con los campos requeridos.

El usuario ingresa los datos del envío. El sistema valida:

- Que no haya campos obligatorios vacíos
- Que las direcciones sean válidas

Si no hay errores, el sistema guarda el envío, asigna un número de guía único y establece su estado inicial como "Pendiente".

SALIDAS:

- Mensaje de confirmación con el número de guía
- Mensaje de error si falta algún campo obligatorio
- Mensaje de error si los datos no cumplen con el formato requerido (por ejemplo, la dirección esté correctamente digitada)

Caso de Uso 3: Asignar Ruta

OBJETIVO: Permitir al administrador o operador asignar una ruta de entrega a uno o varios envíos registrados.

ENTRADAS:

- Número de guía del o envío
- Ruta seleccionada
- Transportista

PROCESOS:

Desde el menú de administración, se accede a la opción “Asignar Ruta”.

El sistema lista los envíos pendientes de asignación.

El usuario selecciona uno o más envíos y luego la ruta correspondiente.

El sistema valida que los envíos estén en estado “Pendiente” y que la ruta esté activa.

Si no hay errores, el sistema asigna la ruta a los envíos seleccionados.

SALIDAS:

- Confirmación de que la ruta fue asignada correctamente
- Mensaje de error si se intenta asignar una ruta a un envío ya asignado
- Mensaje de error si la ruta no está disponible o si hay campos sin completar

Caso de Uso: Ciclo de Vida de un Envío**OBJETIVO:**

Gestionar los estados por los que pasa un envío desde su creación hasta su entrega final.

ENTRADAS:

- Número de guía o ID del envío
- Evento que dispara el cambio de estado (por ejemplo: asignación de ruta, inicio de viaje, confirmación de entrega)

PROCESOS:

1. El usuario registra un nuevo envío.
Estado inicial: Pendiente
2. Un operador o administrador asigna una ruta al envío.
Estado cambia a: En Ruta
3. Durante el proceso de transporte, se pueden registrar actualizaciones de ubicación o estado intermedio.
4. El transportista confirma la entrega al destinatario.
Estado cambia a: Entregado
5. El sistema guarda el historial de todos los cambios de estado para consulta futura.

Validaciones importantes:

- Solo los envíos en estado pendiente pueden pasar a En Ruta.
- Solo los envíos En Ruta pueden pasar a Entregado.
- El sistema no debe permitir saltos de estado inválidos.

SALIDAS:

Confirmación del cambio de estado

Mensaje de error si el cambio no es válido (por ejemplo, cambiar de "Pendiente" a "Entregado" directamente)

Registro en el historial de eventos del envío

ACTORES:

- Operador logístico
- Transportista

- Administrador del sistema

Secuencias: Interacción Usuario -> Angular -> REST API -> Java -> DB.

Esta representa **el flujo completo** de una solicitud en una aplicación web moderna tipo SPA con backend en Java.

1. Usuario → Angular

El usuario interactúa con la aplicación web en el navegador (por ejemplo, hace clic en un botón para consultar un envío).

Angular, que está corriendo en el navegador, detecta este evento y ejecuta una función para llamar al backend.

2. Angular → REST API

Angular hace una **petición HTTP (GET, POST, etc.)** a un **servidor backend** a través de una **REST API**.

Aquí se pasa la información necesaria (como un número de guía, datos del envío, etc.).

3. REST API → Java

El servidor recibe la petición, el servicio determinado REST de Java se encarga de procesar la solicitud por el controlador correspondiente.

4. Java → DB (Base de Datos)

El backend Java accede a la **base de datos** para obtener o modificar datos.

5. La respuesta sube de nuevo

La base de datos devuelve los datos a Java.

Java prepara una respuesta a través del servicio y se la devuelve al cliente formada como un objeto Json Angular.

Angular recibe los datos y **actualiza la interfaz de la cual salió la petición al servicio del backend** y recargar la página.

Resumen Visual

[Usuario (Navegador)]



[Angular SPA (Frontend)]

↓ (HTTP request)

[REST API - Java (Backend)]



[Base de Datos]



[Java responde]



[Angular muestra los datos]

Estados: Ciclo de vida de un Envío (Pendiente → En Ruta → Entregado).

Ciclo de Vida de un Envío

Un envío pasa por varios **estados operativos** desde que se registra hasta que es entregado. A continuación, los principales:

1. Pendiente

Significado: El envío ha sido registrado en el sistema, pero **aún no ha sido asignado a una ruta** ni ha comenzado el proceso de transporte.

Acciones posibles:

Asignar ruta

Editar datos

Cancelar (si aplica)

2. En Ruta

Significado: El envío ya fue asignado a un transportista o ruta de entrega y **está en tránsito** hacia el destinatario.

Puede incluir sub-estados opcionales como:

"En bodega de origen"

"En vehículo"

"En bodega de destino"

Acciones posibles:

Actualizar ubicación

Marcar como "Reprogramado" si hay un retraso

Roles del Sistema

Super -Administrador

- **Habilidades:** Gestion clientes, gestión empleados, asignación de roles, Gestión de usuarios, Gestión Operaciones, configuración del sistema, generación de reportes, generación de informes.
- **Actividades:** Administrar el acceso al sistema, supervisar las operaciones y garantizar la seguridad de la información. Generar, analizar y tomar decisiones en función de los reportes que genere el sistema, carga de datos al sistema

Administrador

- **Habilidades:** Gestion clientes, gestión empleados, asignación de roles, Gestión de usuarios, Gestión Operaciones, generación de reportes, generación de informes.
- **Actividades:** Administrar el acceso al sistema, supervisar las operaciones y garantizar la seguridad de la información. Generar, analizar y tomar decisiones en función de los reportes que genere el sistema, carga de datos al sistema

Operador

- **Habilidades:** Manejo de herramientas de rastreo y optimización de rutas.
- **Actividades:** Uso de aplicaciones de rastreo, seguimiento de envíos, seguimientos de las operaciones, asignación de transportistas, carga de datos al sistema, análisis de posibles

soluciones a las novedades registradas .

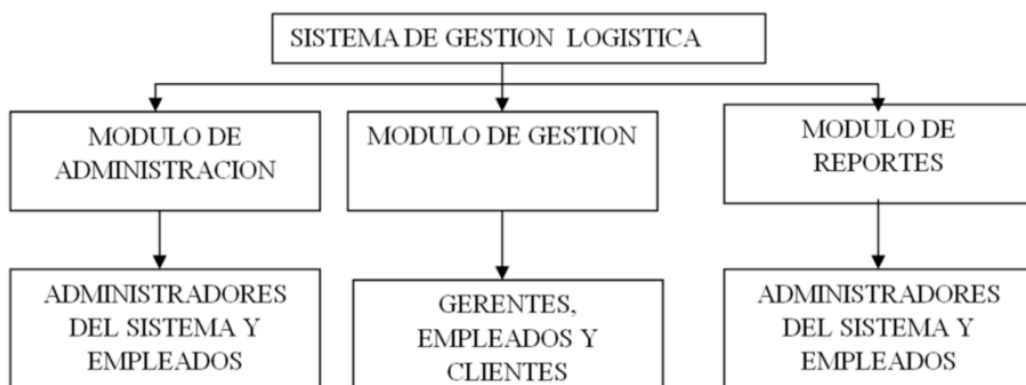
Transportista

- **Habilidades:** Uso de aplicaciones de rastreo y notificación.
- **Actividades:** Transporte de mercancías, actualización de estados de envío, reporte de novedades.

Cliente

- **Habilidades:** Uso básico de la plataforma web/móvil.
- Actividades:** Consulta de envíos, recepción de notificaciones

Diagrama General De Los Módulos del Sistema



Arquitectura del Sistema

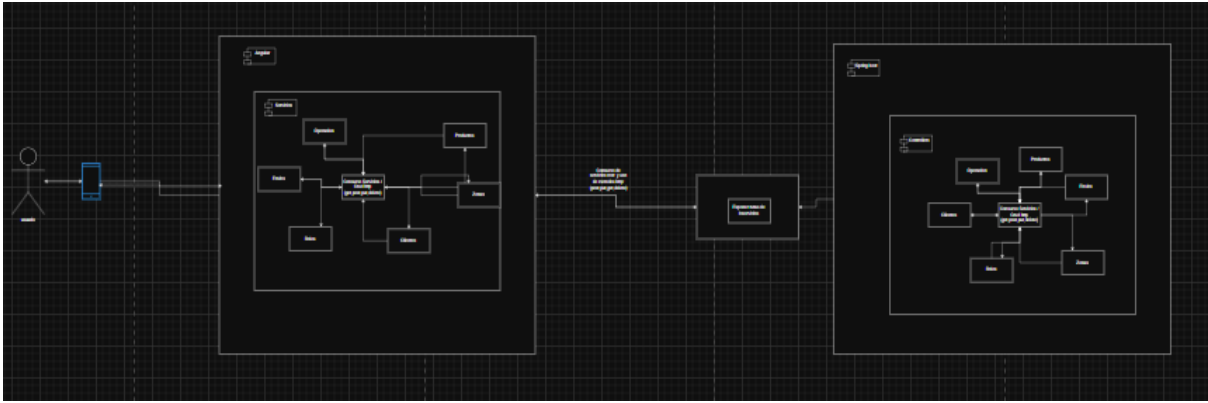


Diagrama de arquitectura Backend

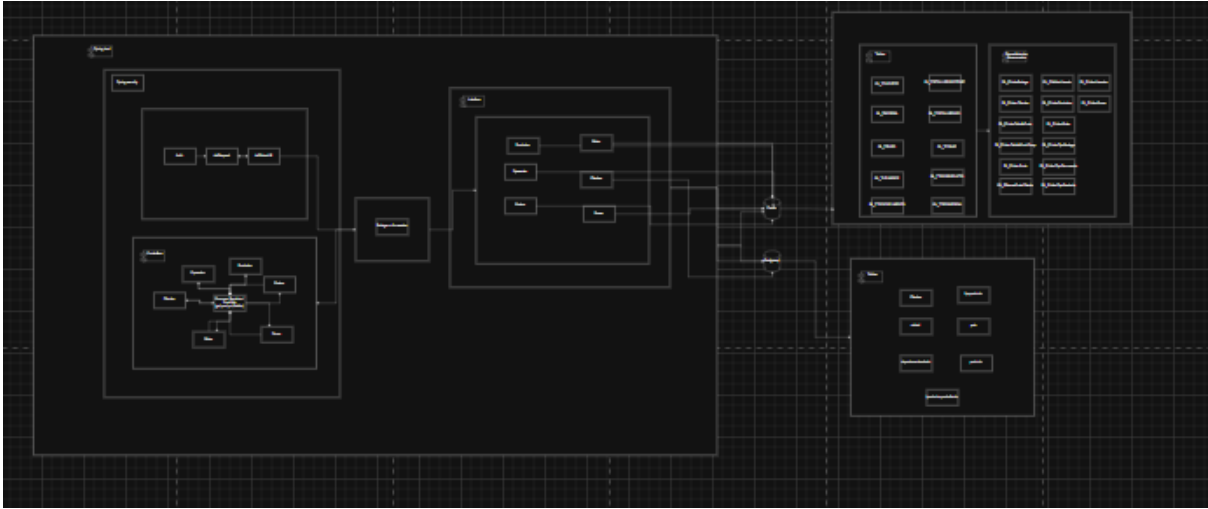
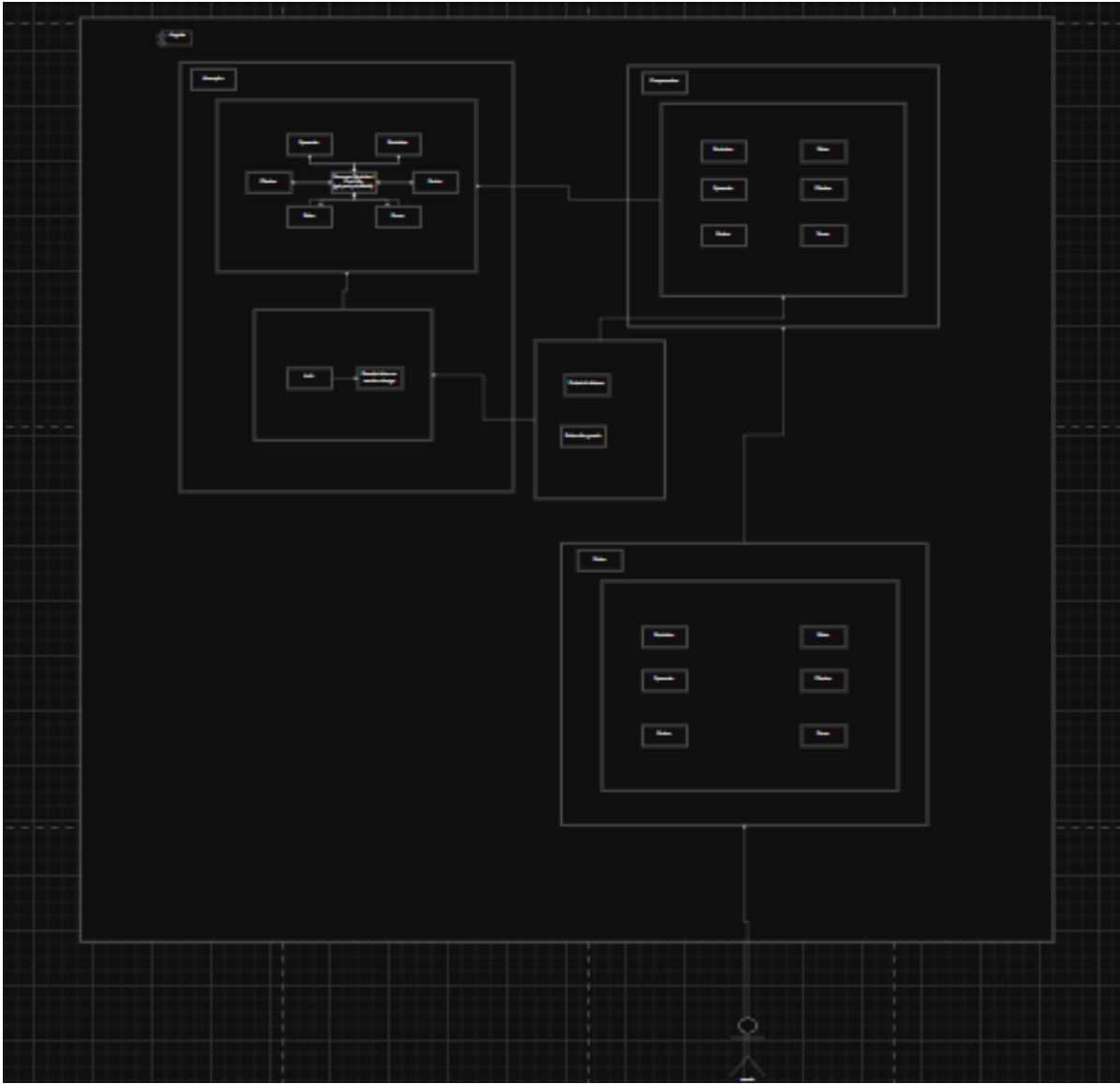
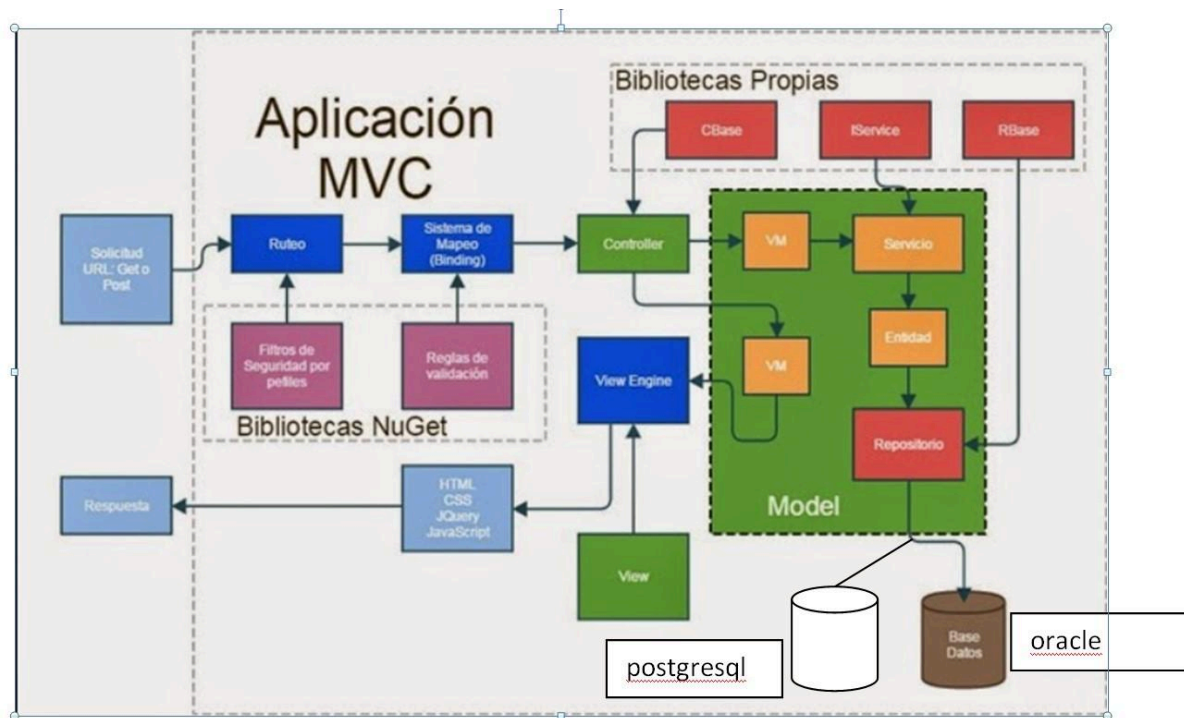


Diagrama de arquitectura FrontEnd



Arquitectura MVC en el backend con Spring Boot.



Componente Función principal

Modelo: Representa los datos y la lógica de negocio.

Vista: Es la interfaz de usuario. Muestra los datos al usuario.

Controlador: Recibe las entradas del usuario y las gestiona.

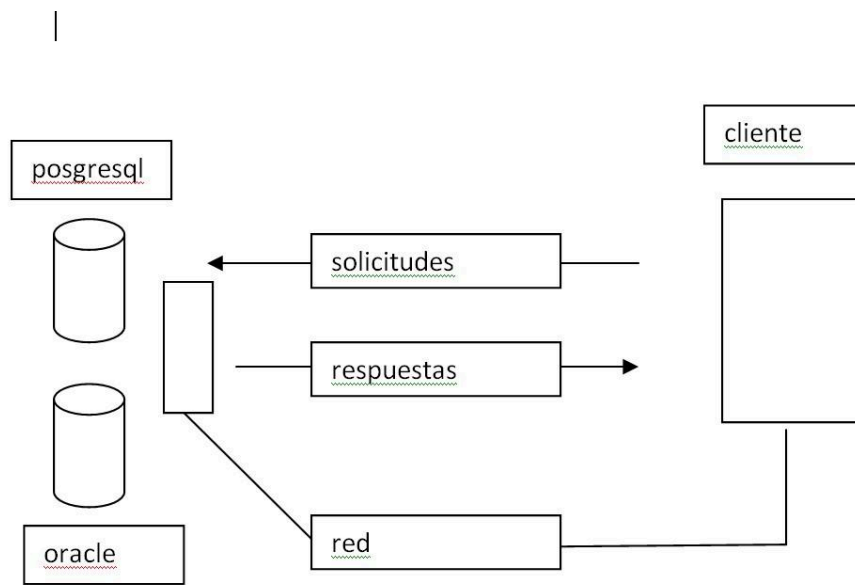
Ejemplo en Spring Boot (MVC Backend):

Modelo: Clases Usuario, Producto, etc. que representan entidades.

Vista: (en aplicaciones con UI en servidor como Thymeleaf). En REST APIs, no suele haber vista. normalmente podemos usar thymeleaf, o podemos usar workflow para el manejo de vista entre el modelo y el controlador

Controlador: Clases con `@Controller` o `@RestController` que manejan las peticiones.

Arquitectura Cliente-servidor.



Cliente: Solicita servicios (generalmente una aplicación web o móvil).

Servidor: Proporciona esos servicios (por ejemplo, procesamiento de datos, acceso a base de datos).

Características:

Comunicación generalmente a través de HTTP/HTTPS.

Separación clara entre frontend y backend.

Muy usada en aplicaciones web y móviles modernas.

Ejemplo con Angular y Spring Boot:

Cliente: Aplicación Angular que se ejecuta en el navegador.

Servidor: API REST construida con Spring Boot que procesa solicitudes y accede a la base de datos.

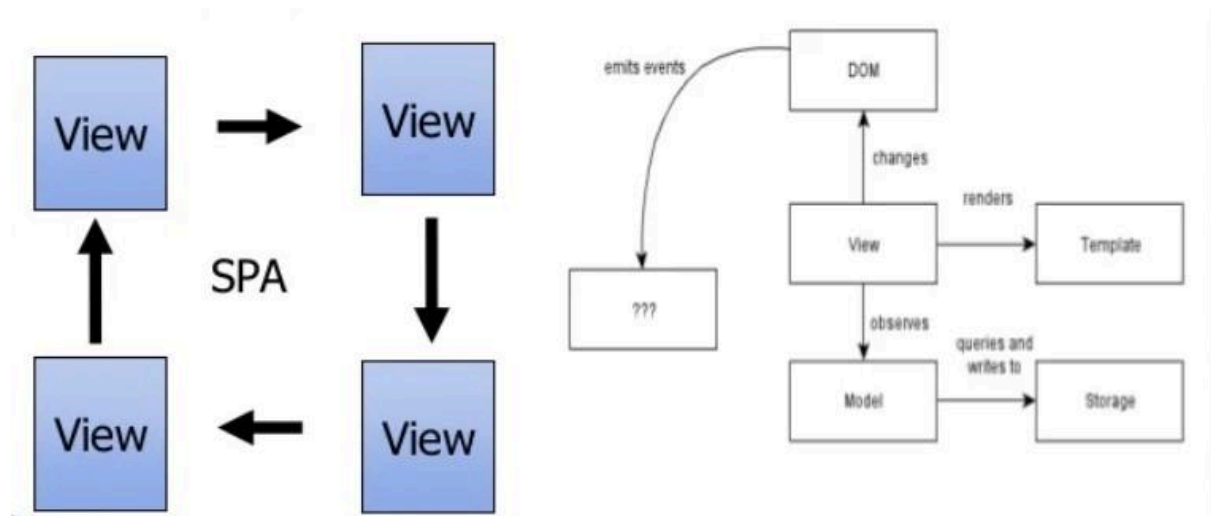
Ventajas:

Escalabilidad: puedes escalar el cliente y el servidor por separado.

Mantenimiento: separación de responsabilidades.

Reusabilidad del backend: distintos clientes (web, móvil) pueden usar la misma API.

Como funciona ... Sigle page application en Angular



Carga inicial única (index.html):

Angular carga una única página HTML (index.html) cuando entras a la app. Esta página contiene poco más que un `<app-root>` (el componente raíz).

Routing sin recarga de página:

Angular usa su propio enrutador (Router) para navegar entre diferentes "vistas" o componentes sin que el navegador recargue la página.

Ejemplo: navegar de inicio a productos no recarga la página; Angular simplemente cambia el componente que se muestra.

Renderizado dinámico con componentes:

El contenido se organiza en componentes. Cuando cambias de ruta, Angular muestra el componente correspondiente dentro del `<router-outlet>`.

API en segundo plano (Backend):

La aplicación consume datos a través de peticiones HTTP (como `HttpClient`) a APIs externas o servicios propios, y actualiza la vista dinámicamente.

Recursos estáticos cargados una vez:

Todos los recursos (HTML, JS, CSS) se cargan una vez, y luego Angular maneja la interacción del usuario sin nuevas cargas desde el servidor.

Explicación de Patrones de diseño Usados Repositorio, Service, Controller.

Back end

Patron Repository (Repositorio)

Propósito: Acceder a la base de datos.

Qué hace: Realiza operaciones CRUD (Create, Read, Update, Delete) sobre las entidades.

Tecnología en Spring: Se define como una interfaz que extiende JpaRepository o CrudRepository.

Patron Service (Servicio)

Propósito: Contiene la lógica de negocio.

Qué hace: Coordina y aplica reglas sobre los datos que vienen del repositorio antes de pasarlos al controlador.

Ventaja: Separa la lógica de negocio del controlador y del acceso a datos.

Patron Controller (Controlador)

Propósito: Maneja las peticiones HTTP.

Qué hace: Actúa como intermediario entre el cliente (Angular) y el backend (servicio).

Tecnología en Spring: Anotado con **@RestController**

Front end

Service (Servicio)

Propósito: Encapsula llamadas HTTP y lógica compartida.

Qué hace: Realiza peticiones al backend (Spring Boot), maneja la comunicación.

Tecnología: Usa HttpClient.

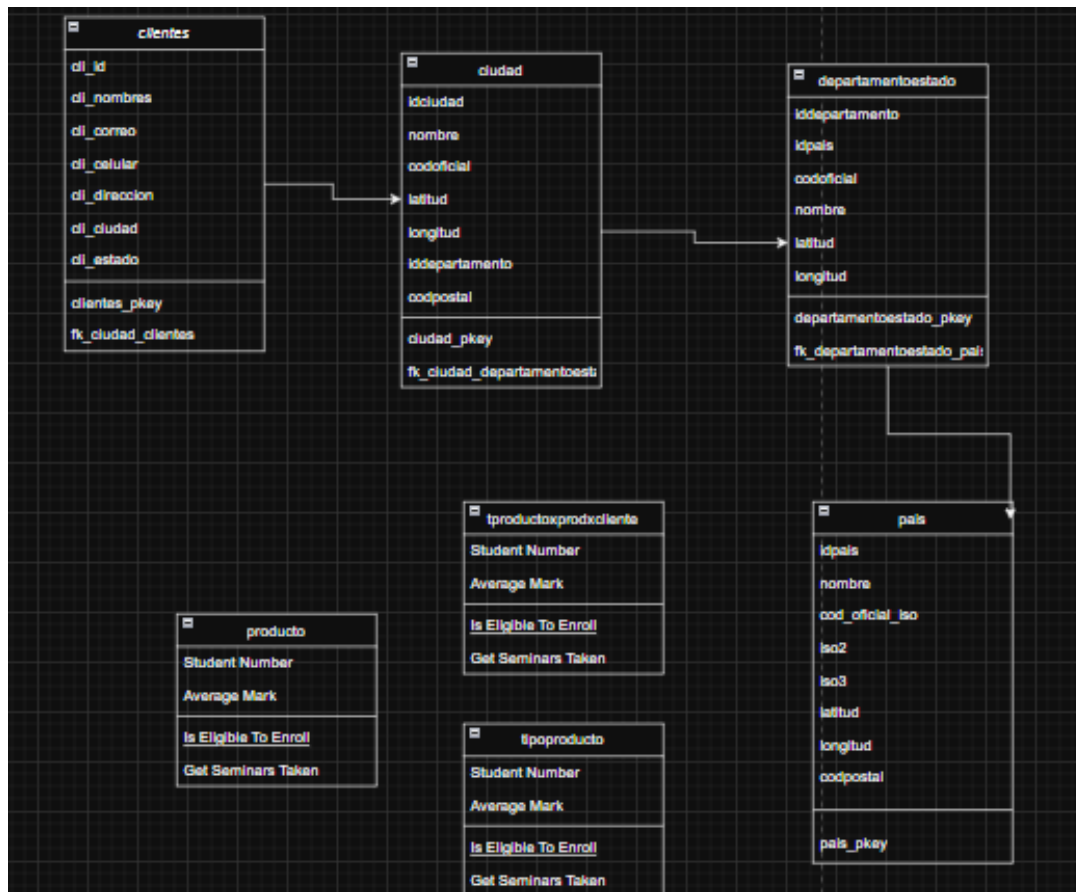
Component (Componente)

Propósito: Muestra la interfaz de usuario.

Qué hace: Llama a los servicios y muestra datos.

Tecnología: HTML + TypeScript (decorado con @Component).

Oracle:



Postgresql

Frontend (Angular):

- **Formulario con selects para (Bodega, Ciudad Destino, Ciudad Origen, Mercancia, Estado)**
- **Validaciones (Bodega obligatorio, origen y destino, cliente)**
- **Servicio envio.service.ts que llama a la API con HttpClient**
- **Vistas/envios que llama a un dialog nuevo-envio.component.ts**

Funcionalidad 2: Listar Envíos

Permite Listar todos los envío indicando su tipo (aéreo, marítimo o terrestre), origen, destino, peso, dimensiones, transportista y usuario responsable.

EndPoint

- **Frameworks: Spring Boot + Angular**
- **Base de datos: PostgreSQL y oracle**

Frontend (Angular):

- **Formulario Lista los principales detalles de los envíos Validaciones (Bodega**
- **Servicio envio.service.ts que llama a la API con HttpClient**
- **Vistas/envios que llama a el componente de listar envios envios.component.ts**

Pruebas Realizadas

Pruebas Unitarias:

Las pruebas unitarias son un tipo de pruebas que se enfocan en verificar el correcto funcionamiento de la unidad más pequeña del código, normalmente se aplica sobre funciones o métodos de forma aislada. Las pruebas unitarias nos sirven para detectar errores en funciones específicas, verificar que el código haga lo que se espera.

JUnit para backend.

Las pruebas unitarias se enfocan en validar el comportamiento de métodos individuales. Se utilizó **JUnit 5** junto con **Mockito** para simular dependencias del sistema.

1. Pruebas de Integración

Caso de prueba: Crear envío exitoso

Objetivo:

Verificar que el endpoint REST `/crearEnvio` acepte un objeto `Envio` válido, invoque correctamente el servicio para crear el envío y retorne una respuesta exitosa con el código 200 y el mensaje esperado.

Descripción del caso:

Se simula el comportamiento del servicio `servicioEnvio.crearEnvio(envio)` para que, cuando se le envíe un objeto `Envio` con datos válidos, retorne un mensaje JSON indicando que el registro fue exitoso. Luego, se realiza una llamada HTTP POST al endpoint `/crearEnvio` enviando el objeto `Envio` en formato JSON. Finalmente, se verifica que el estado HTTP de la respuesta sea 200 (OK) y que el contenido devuelto coincida con el mensaje simulado por el servicio.

Pasos realizados en el test:

Preparación:

Se crea un objeto `Envio` con datos válidos (`envioMock`), que contiene campos como ID, teléfono, fechas, ciudades origen y destino, estado y referencia.

Se define la respuesta esperada del servicio: un JSON con

```
{"codigo": "200", "mensaje": "Registro exitoso"}
```

Mock del servicio:

Se usa Mockito para simular el método `crearEnvio(envioMock)` y que retorne la respuesta esperada sin ejecutar la lógica real del servicio.

Ejecución de la petición:

Se utiliza MockMvc para realizar una petición POST al endpoint `/crearEnvio`.

El cuerpo de la petición es el objeto `Envio` convertido a JSON usando `objectMapper`.

Se especifica que el contenido es JSON y se acepta respuesta JSON.

Verificación de resultados:

Se comprueba que el código de estado HTTP sea 200 (OK).

Se valida que el cuerpo de la respuesta coincida exactamente con la cadena JSON simulada.

Resultado:

La aplicación devuelve una respuesta con código HTTP 200

Caso de prueba: Listar detalle de usuarios

Objetivo:

Verificar que el endpoint REST encargado de listar los usuarios (`/tcc/gestion/CrudUsuarios/ListarUsuarios`) responda correctamente con un estado HTTP 200 (OK) y que el servicio que obtiene la lista de usuarios sea invocado y funcione como se espera.

Descripción del caso:

Se crea una lista simulada (mock) de objetos `Usuarios` con datos ficticios que representan usuarios registrados en el sistema. Luego, se usa Mockito para que cuando se invoque el método `listadoUsuarios()` del servicio `servicioUsuario`, retorne esta lista simulada. Después, se realiza una petición HTTP GET al endpoint que lista los usuarios y se valida que la respuesta tenga un código 200, indicando que la solicitud fue exitosa.

Pasos realizados en el test:

Preparación del mock:

Se crea una lista llamada `usuariosMock` con dos objetos `Usuarios` que contienen datos de ejemplo (ID, nombre, dirección, teléfono, correo, etc.).

Simulación del servicio:

Se utiliza Mockito para definir que cuando se llame a `servicioUsuario.listadoUsuarios()`, este método retorne la lista `usuariosMock` simulada.

Ejecución de la petición:

Se realiza una petición HTTP GET al endpoint `/tcc/gestion/CrudUsuarios/ListarUsuarios` usando `MockMvc`.
Se indica que se espera una respuesta en formato JSON.

Verificación de la respuesta:

Se obtiene la respuesta HTTP del servidor.
Se comprueba que el código de estado HTTP sea 200 (OK), lo que indica que la petición fue procesada correctamente.

Manejo de excepciones:

Si ocurre alguna excepción durante la ejecución del test, esta se registra en el log para diagnóstico,

evitando que el test termine abruptamente sin información.

Resultado :

El servidor respondió con un estado HTTP 200 (OK) cuando se realiza la petición GET para listar usuarios, y el servicio retorno correctamente la lista simulada de usuarios.

```
@Test
void testListarDetalleUsuarios() throws SQLException {

    List<Usuarios> usuariosMock = Arrays.asList(
        new Usuarios(1, "Juan", "calle falsa 123", "3164567212", "jgabriel@gmail.com",
            "41562258", "Admin123", 1, 1, 1),
        new Usuarios(1, "Maria", "calle 4 # 12 - 123", "3104567010", "maria@gmail.com",
            "170458962", "Admin1234-1", 1, 2, 1)
    );

    when(servicioUsuario.listadoUsuarios()).thenReturn(usuariosMock);

    MockHttpServletResponse response;
    try {
        response = mockMvc.perform(get("/tcc/gestion/CrudUsuarios/ListarUsuarios")
            .accept(MediaType.APPLICATION_JSON))
            .andReturn().getResponse();

        assertThat(response.getStatus()).isEqualTo(HttpStatus.OK.value());
    } catch (Exception e) {
        log.error(ExceptionUtil.format(e));
    }
}
```

2. Pruebas Automatizadas (JUnit + Mockito)

Caso de prueba: Buscar usuario por ID

Objetivo:

Verificar que el endpoint REST para buscar un usuario por su ID ([/tcc/gestion/CrudUsuarios/buscarUsuariosById](#)) funcione correctamente, invocando el servicio que realiza la búsqueda y retornando una respuesta adecuada.

Descripción del caso:

Se simula la respuesta del servicio [servicioUsuario](#) para el método

`buscarUsuariosXId(1)`, de modo que retorne un objeto `Usuarios` con datos predeterminados. Luego, se realiza una petición HTTP GET al endpoint correspondiente con el parámetro `id=1`. Finalmente, se verifica que el servicio haya sido llamado con el ID correcto.

Pasos realizados en el test:

Preparación del mock:

Se crea un objeto `Usuarios` con datos ficticios (ID, nombre, dirección, teléfono, correo, etc.) que representa al usuario buscado.

Simulación del servicio:

Se configura Mockito para que cuando se llame al método `servicioUsuario.buscarUsuariosXId(1)`, retorne el usuario mock creado.

Ejecución de la petición:

Se realiza una petición HTTP GET a la ruta `/tcc/gestion/CrudUsuarios/buscarUsuariosById` pasando el parámetro `id` con valor `"1"`.

Se indica que se espera una respuesta en formato JSON.

Verificación de interacción:

Se verifica que el método `buscarUsuariosXId(1)` haya sido efectivamente llamado en el servicio.

Manejo de excepciones:

Si ocurre alguna excepción durante la ejecución, se registra en el log para facilitar el diagnóstico.

Resultado :

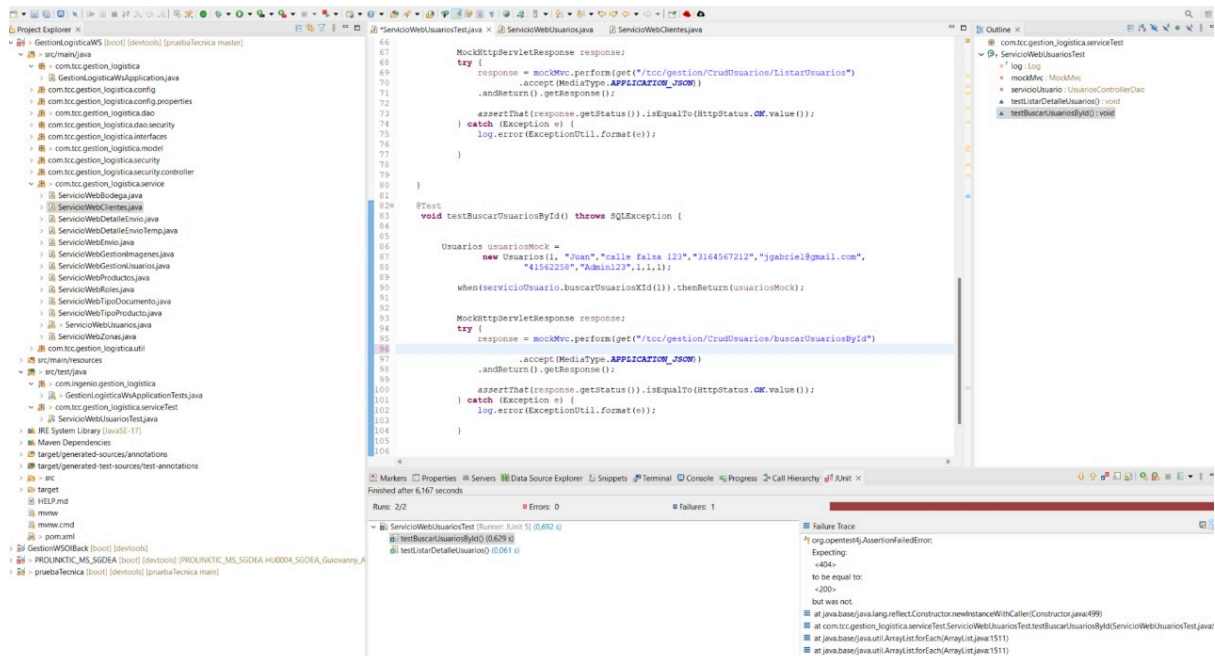
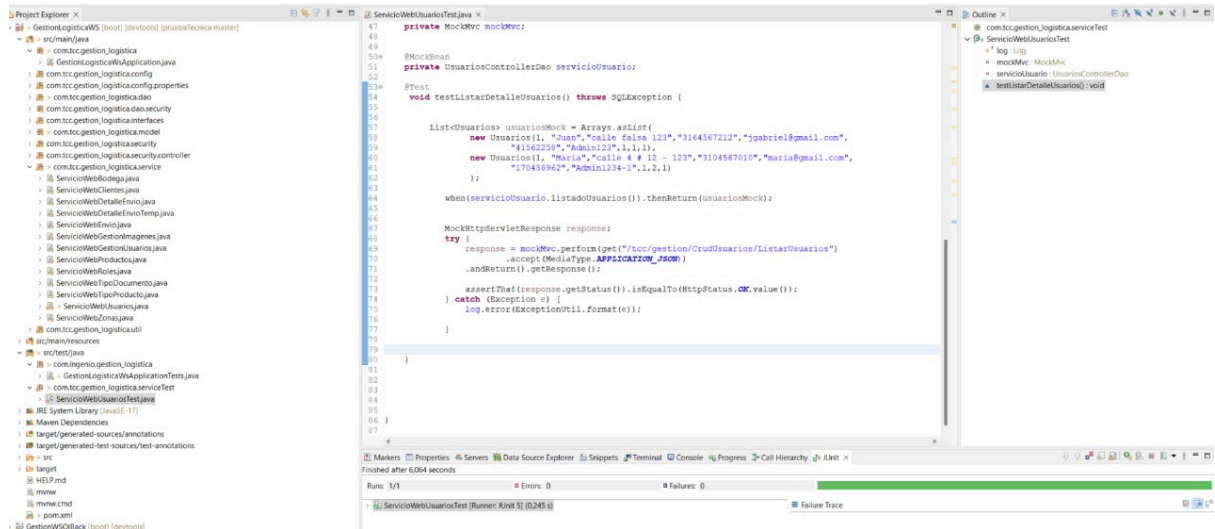
El servicio recibe la llamada para buscar el usuario con ID 1 y devuelve el usuario mock correspondiente, y la respuesta HTTP es correcta (usualmente 200 OK aunque aquí no se valida explícitamente).

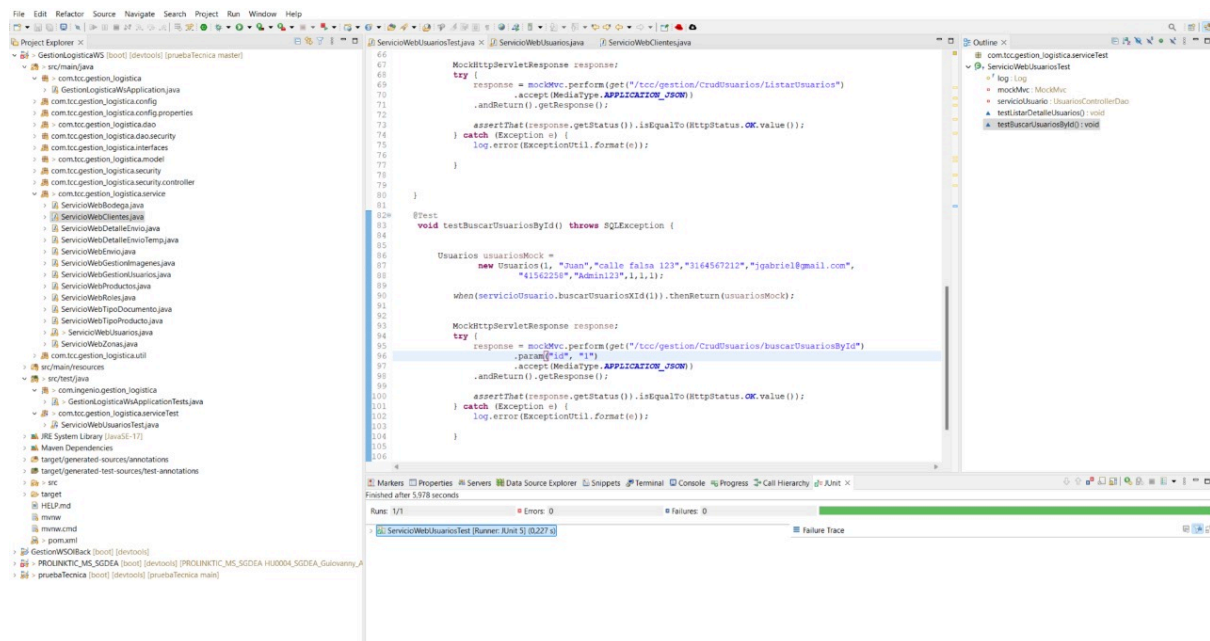
```

84      @Test
85      void testBuscarUsuariosById() throws SQLException {
86
87
88          Usuarios usuariosMock =
89              new Usuarios(1, "Juan", "calle falsa 123", "3164567212", "jgabriel@gmail.com",
90                          "41562258", "Admin123", 1, 1, 1);
91
92          when(servicioUsuario.buscarUsuariosXId(1)).thenReturn(usuarioMock);
93
94
95          MockHttpServletResponse response;
96          try {
97              response = mockMvc.perform(get("/tcc/gestion/CrudUsuarios/buscarUsuariosById")
98                                      .param("id", "1")
99                                      .accept(MediaType.APPLICATION_JSON))
100                             .andReturn().getResponse();
101              verify(servicioUsuario).buscarUsuariosXId(1);
102
103
104          } catch (Exception e) {
105              log.error(ExceptionUtil.format(e));
106
107          }
108
109
110      }

```

Aquí está la ejecución de las pruebas:





Pruebas de Usabilidad: Interacción con la API

Objetivo

Evaluar si el sistema es intuitivo para el usuario final al interactuar con los formularios o *endpoints* desde Swagger UI o Postman.

Procedimiento

Para esta prueba, se seleccionaron tres usuarios clave con experiencia en la gestión de pedidos y logística. Se les solicitó que realizaran tareas específicas utilizando los *endpoints* a través de Swagger UI, simulando escenarios operativos reales. Una de las tareas fue la creación de un nuevo registro de cliente, donde debían ingresar información como el nombre, la dirección y el número de teléfono. Durante la ejecución de las tareas, los evaluadores observaron el comportamiento de los usuarios y registraron su *feedback* sobre la comprensibilidad de los mensajes del sistema.

Conclusiones

Los usuarios lograron comprender de manera efectiva tanto los mensajes de éxito (por ejemplo, **200 OK - Cliente creado exitosamente**) como los mensajes de error generales (por ejemplo, **400 Bad Request - Error de datos**). Sin embargo, se identificó un área de mejora significativa en la especificidad de los mensajes de error de validación de entrada.

Ejemplo de Mejora Propuesta:

Situación Actual: Cuando un usuario intentaba ingresar un número de teléfono en un formato incorrecto (ej. "310-123-4567" o "ABCDE"), el sistema retornaba un mensaje genérico como:

HTTP/1.1 400 Bad Request

Content-Type: application/json

```
{  
  "code": "VALIDATION_ERROR",  
  "message": "Formato de teléfono inválido."  
}
```

Los usuarios reportaron que, aunque sabían que era un error de formato, no les quedaba claro qué formato era el esperado, lo que los obligaba a adivinar o a recurrir a la documentación.

Sugerencia de Mejora: Se propuso que, para errores de validación de formato, el mensaje fuera mucho más explícito y guiador, indicando el formato correcto esperado. Por ejemplo:

HTTP/1.1 400 Bad Request

Content-Type: application/json

```
{  
  "code": "PHONE_FORMAT_INVALID",  
  "message": "El número de teléfono debe contener exactamente 10 dígitos numéricos  
sin guiones ni espacios. Ejemplo: 3101234567."  
}
```

Esta mejora se considera crítica para reducir la frustración del usuario, minimizar los errores de entrada y agilizar la operación diaria en el sistema de gestión logística, impactando directamente en la eficiencia del personal de *back-office* que interactúa con el sistema.

Manual Técnico

Descripción General del Proyecto

Las empresas dedicadas al transporte de mercancías enfrentan retos constantes en la gestión eficiente de sus operaciones logísticas. En particular, la empresa *** (nombre reservado), especializada en transporte marítimo, terrestre y aéreo, ha identificado múltiples problemas relacionados con el extravío, daño o retraso de productos durante el trayecto. Estas situaciones no solo generan pérdidas económicas, sino que también afectan negativamente su reputación y credibilidad ante los clientes.

Para hacer frente a estas dificultades, se ha decidido desarrollar e implementar un sistema de software especializado para la gestión y monitoreo de envíos, con el fin de garantizar un control total de la mercancía desde su origen hasta el destino final.

Este sistema permitirá a la empresa realizar un seguimiento en tiempo real de los vehículos mediante tecnologías de geolocalización (GPS), registrar los movimientos de la mercancía en las bodegas, y gestionar de forma centralizada toda la información logística relevante, incluyendo el vehículo encargado del traslado y la bodega de recepción.

Además, se incorpora un módulo de seguridad para proteger la información de los envíos y un sistema de alertas y notificaciones ante retrasos, desvíos o incidentes con la carga.

Arquitectura del Sistema

La aplicación se desarrolló siguiendo el patrón de arquitectura en capas, que permite una organización clara, escalable y mantenible del código. Esta arquitectura divide la aplicación en capas funcionales, cada una con una responsabilidad específica, facilitando así la separación de preocupaciones, la reutilización del código y la posibilidad de realizar pruebas de manera más efectiva.

Capa de Controladores (Service)

Responsabilidad: Esta capa se encarga de recibir y gestionar las solicitudes HTTP realizadas por los clientes (navegadores, aplicaciones móviles, otros servicios, etc.).

Ubicación: `com.tcc.gestion_logistica.service`

Características:

Contiene los *endpoints* RESTful.

Valida la información de entrada básica (por ejemplo, datos obligatorios).

Delegan la lógica de negocio a la capa de servicio.

Devuelven respuestas adecuadas al cliente (códigos HTTP y cuerpos JSON)

Capa de Seguridad (Security)

Responsabilidad: Gestiona el control de acceso a los recursos del sistema. Esta capa se encarga de autenticar a los usuarios (verificar su identidad) y autorizar sus acciones (verificar sus permisos) según roles definidos. Su función principal es proteger la integridad, confidencialidad y disponibilidad de la aplicación.

Ubicación: `com.tcc.gestion_logistica.security`

Características:

Autenticación de usuarios: Verifica que las credenciales ingresadas por un usuario correspondan a una cuenta válida en el sistema.

Autorización basada en roles: Controla a qué recursos puede acceder un usuario dependiendo de su rol (ejemplo: `ADMIN`, `OPERADOR`, `CLIENTE`).

Protección de *endpoints*: Define qué rutas son públicas, protegidas o requieren roles específicos.

Integración con Spring Security: Se utiliza `SecurityFilterChain` o `WebSecurityConfigurerAdapter` (según la versión de Spring Boot) para definir las reglas de seguridad.

Cifrado de contraseñas: Utiliza `PasswordEncoder` para proteger las contraseñas almacenadas en la base de datos.

Gestión de sesiones o *tokens* JWT: Soporta autenticación con sesión tradicional o sin estado (*stateless*) usando JSON Web Tokens (JWT) para la comunicación segura.

Configuración CORS: Permite o restringe el acceso al *backend* desde ciertos dominios para evitar problemas de seguridad entre orígenes.

Relación con otras capas:

Con la capa Service: Protege los *endpoints* del controlador con reglas de acceso, asegurando que solo usuarios autorizados puedan invocar ciertas funcionalidades.

Con la capa DAO: Usa el `UsuariosDao` para obtener usuarios desde la base de datos, validar sus credenciales y convertirlos en objetos autenticables para Spring Security.

Capa de Acceso a Datos (DAO o Repository)

Responsabilidad: Provee una abstracción para acceder a la base de datos. Gestiona todas las operaciones CRUD (Create, Read, Update, Delete) sobre las entidades del negocio, aislando la lógica de negocio de los detalles de persistencia.

Ubicación: `com.tcc.gestion_logistica.dao`

Características:

Utiliza una fábrica de conexiones o un ORM (Object-Relational Mapping, como Hibernate/JPA) para comunicarse directamente con la base de datos.

Su implementación puede ser fácilmente intercambiable o simulada (*mocked*) en pruebas unitarias e de integración.

Define interfaces (`IEnvioDAO`, `IUsuarioDAO`, etc.) para desacoplar la implementación concreta.

Capa de Modelos o Entidades (Model)

Responsabilidad: Define las estructuras de datos que representan las entidades del negocio y su mapeo con la base de datos. Estas clases se usan tanto para la persistencia de datos como para el transporte de información entre las distintas capas de la aplicación.

Ubicación: `com.tcc.gestion_logistica.model`

Características:

Anotadas con `@Entity` según sea el caso, para el mapeo ORM.

Usadas tanto para la definición del esquema de la base de datos como para la representación de los datos en las respuestas de la API.

Pueden incluir anotaciones de validación (por ejemplo, `@NotNull`, `@Size`, `@Pattern`, etc.) para asegurar la integridad de los datos antes de su procesamiento o persistencia.

Capa de Configuración (Config)

Responsabilidad: Contiene todas las clases que definen parámetros y comportamientos de configuración global para la aplicación. Esta capa permite establecer cómo se conectan y comportan los distintos módulos del sistema, centralizando la gestión de configuraciones.

Ubicación: `com.tcc.gestion_logistica.config`

Características:

Define la configuración de:

CORS (Cross-Origin Resource Sharing): Reglas para permitir o denegar solicitudes de dominios cruzados.

Swagger/OpenAPI: Configuración para la generación automática de la documentación del API REST.

Propiedades de conexión a la base de datos: Detalles como URL, usuario, contraseña, *driver*, y *pool* de conexiones.

Configuración de seguridad: Parámetros específicos para Spring Security.

Es altamente reutilizable y centraliza parámetros críticos del sistema, facilitando su modificación y mantenimiento.

Capa de Utilidades (Util)

Responsabilidad: Agrupa clases auxiliares y herramientas reutilizables que apoyan distintas operaciones comunes en la lógica de negocio, controladores, servicios o manejo de excepciones. Su objetivo es reducir la duplicación de código y promover la modularidad.

Ubicación: `com.tcc.gestion_logistica.util`

Características:

Contiene clases estáticas o con métodos reutilizables que no pertenecen directamente a la lógica de negocio principal.

Facilita la realización de operaciones comunes como formateo de datos, validaciones genéricas, manejo de fechas, etc.

Tiene dependencia directa de otras capas, lo que facilita su uso en cualquier parte del proyecto donde se necesite una funcionalidad auxiliar.

Estructura del Proyecto

```

└─ src/main/java
  └─ com.tcc.gestion_logistica
    └─ com.tcc.gestion_logistica.config
      └─ com.tcc.gestion_logistica.config.properties
    └─ com.tcc.gestion_logistica.dao
      └─ com.tcc.gestion_logistica.dao.security
    └─ com.tcc.gestion_logistica.interfaces
    └─ com.tcc.gestion_logistica.model
    └─ com.tcc.gestion_logistica.security
      └─ com.tcc.gestion_logistica.security.controller
    └─ com.tcc.gestion_logistica.service
    └─ com.tcc.gestion_logistica.util
  └─ src/main/resources
  └─ src/test/java
    └─ com.ingenio.gestion_logistica
      └─ com.tcc.gestion_logistica.serviceTest
        └─ ServicioWebUsuariosTest.java
        └─ ServicioWebEnviosTest.java
```

Explicación

```
├── dao/                                // Capa de Acceso a Datos
│   ├── interfaces/                    // Interfaces de los DAOs
│   │   ├── IEnvioDAO.java
│   │   ├── IUsuarioDAO.java
│   │   ├── IVehiculoDAO.java
│   │   └── IBodegaDAO.java
│   └── implementations/              // Implementaciones concretas de los DAOs
│       ├── EnvioDAOImpl.java
│       ├── UsuarioDAOImpl.java
│       ├── VehiculoDAOImpl.java
│       └── BodegaDAOImpl.java
├── model/                             // Capa de Modelos/Entidades
│   ├── Envio.java
│   ├── Usuarios.java
│   ├── Vehiculo.java
│   ├── Bodega.java
│   ├── Mercancia.java
│   └── Alerta.java
├── security/                          // Capa de Seguridad
│   ├── JwtRequestFilter.java         // Filtro para validar JWT
│   ├── JwtUtil.java                 // Utilidades para JWT
│   ├── ServicioSeguridadController.java // Endpoint de autenticación y seguridad
│   └── UserDetailsServiceImpl.java   // Implementación de UserDetailsService para Spring Security
├── service/                           // Capa de Servicios (incluye Controladores)
│   ├── controller/                  // Controladores RESTful
│   │   ├── EnvioController.java
│   │   ├── UsuarioController.java
│   │   ├── VehiculoController.java
│   │   └── BodegaController.java
│   ├── EnvioService.java            // Lógica de negocio para Envíos
│   ├── UsuarioService.java          // Lógica de negocio para Usuarios
│   ├── ServicioZonas.java           // Lógica de negocio para Zonas (geográficas)
│   ├── VehiculoService.java         // Lógica de negocio para Vehículos
│   └── BodegaService.java           // Lógica de negocio para Bodegas
└── util/                             // Capa de Utilidades
    ├── DateUtil.java               // Utilidades para manejo de fechas
    ├── ResponseUtil.java           // Utilidades para estandarizar respuestas API
    ├── ValidationUtil.java         // Utilidades para validaciones genéricas
    └── GeolocalizacionUtil.java    // Utilidades para procesamiento de datos de geolocalización
```

Explicación por Módulo

A continuación, se detalla la funcionalidad y las responsabilidades específicas de las clases clave dentro de cada módulo:

config/

Este módulo centraliza la configuración de la aplicación.

DatabaseConfig.java: Define la configuración de la conexión a la base de datos, incluyendo la fábrica de conexiones que lee los parámetros desde un archivo de propiedades (ej.

database.properties) para establecer y gestionar las conexiones.

SecurityConfig.java: Configura las reglas de seguridad de Spring Security, definiendo

patrones de URL protegidos, roles de acceso, y el mecanismo de autenticación (por ejemplo, JWT).

SwaggerConfig.java: Habilita y configura la documentación de la API REST utilizando Swagger/OpenAPI, permitiendo la exploración interactiva de los *endpoints*.

dao/

Este módulo se encarga de la interacción directa con la base de datos.

interfaces/: Contiene las interfaces que definen los contratos para las operaciones CRUD de cada entidad.

- **IEnvioDAO**: Interfaz para operaciones de persistencia de envíos.
- **IUsuarioDAO**: Interfaz para operaciones de persistencia de usuarios.
- **IVehiculoDAO**: Interfaz para operaciones de persistencia de vehículos.
- **IBodegaDAO**: Interfaz para operaciones de persistencia de bodegas.

implementations/: Contiene las clases que implementan las interfaces DAO, utilizando la fábrica de conexiones para ejecutar sentencias SQL directamente contra la base de datos.

- **EnvioDAOImpl**: Implementación de **IEnvioDAO**.
- **UsuarioDAOImpl**: Implementación de **IUsuarioDAO**.
- **VehiculoDAOImpl**: Implementación de **IVehiculoDAO**.
- **BodegaDAOImpl**: Implementación de **IBodegaDAO**.

model/

Define las entidades del negocio.

Envio.java: Representa los datos de un envío, incluyendo **idEnvio**, **origen**, **destino**, **estado** (en tránsito, entregado, dañado, extraviado), **fechaEnvio**, **fechaEntregaEstimada**, **vehiculoAsignado**, **bodegaRecepcion**, **mercancia** (referencia a la carga), **seguridad** (módulo de seguridad asociado), y **alertas** (lista de alertas generadas).

Usuarios.java: Representa los datos del usuario del sistema, como **idUsuario**, **nombre**, **apellido**, **email**, **telefono**, **rol** (ADMIN, OPERADOR, CLIENTE), **username**, y **password** (cifrada).

Vehiculo.java: Define las propiedades de un vehículo de transporte, incluyendo **idVehiculo**, **tipo** (camión, barco, avión), **placa**, **capacidad** (en peso/volumen), **estado** (disponible, en ruta, en mantenimiento), y **ubicacionActual** (coordenadas GPS).

Bodega.java: Representa una bodega o almacén, con **idBodega**, **nombre**, **direccion**, **capacidad** y **ubicacion** (geográfica).

Mercancia.java: Describe la carga específica de un envío, con **idMercancia**,

`descripcion`, `peso`, `volumen` y `estado` (ej. "intacto", "dañado").

`Alerta.java`: Representa una notificación o incidente relacionado con un envío, con `idAlerta`, `tipoAlerta` (retraso, desvío, daño), `descripcion`, `fechaHora`, `estado` (activa, resuelta) y `idEnvio` (referencia al envío afectado).

security/

Gestiona la autenticación y autorización de usuarios.

`ServicioSeguridadController.java`: Expone los *endpoints* relacionados con la autenticación de usuarios (ej. `/auth/login` para iniciar sesión y generar *tokens* JWT) y, si aplica, el registro de nuevos usuarios o la gestión de *tokens*.

`JwtUtil.java`: Clase auxiliar para la creación, validación y extracción de información de los *tokens* JWT.

`UserDetailsServiceImpl.java`: Implementación de la interfaz `UserDetailsService` de Spring Security, utilizada para cargar los detalles del usuario a partir de su nombre de usuario durante el proceso de autenticación.

`JwtRequestFilter.java`: Un filtro de Spring Security que intercepta las solicitudes HTTP, extrae y valida el *token* JWT del encabezado de autorización, y establece el contexto de seguridad del usuario autenticado.

service/

Contiene la lógica de negocio y los controladores REST.

controller/:

`EnvioController.java`: Expone los *endpoints* REST para la gestión de envíos (ej. `/api/envios`, `/api/envios/{id}`), permitiendo operaciones CRUD y la consulta del estado de los envíos.

`UsuarioController.java`: Expone los *endpoints* REST para la gestión de usuarios (ej. `/api/usuarios`, `/api/usuarios/{id}`), incluyendo la búsqueda por ID o el listado completo.

`VehiculoController.java`: Expone los *endpoints* REST para la gestión de vehículos (ej. `/api/vehiculos`, `/api/vehiculos/{id}`), permitiendo registrar, actualizar y consultar información de vehículos.

`BodegaController.java`: Expone los *endpoints* REST para la gestión de bodegas (ej. `/api/bodegas`, `/api/bodegas/{id}`), permitiendo registrar, actualizar y consultar información de bodegas e inventario.

EnvioService.java: Contiene la lógica de negocio para registrar un nuevo envío, actualizar su estado, asignar vehículos, gestionar alertas relacionadas, y realizar consultas complejas sobre envíos.

UsuarioService.java: Implementa la lógica de negocio para operaciones sobre usuarios, como registro, búsqueda por ID, listado completo, actualización de perfiles y eliminación.

ServicioZonas.java: Encargado de la lógica relacionada con la gestión de zonas geográficas. Podría incluir métodos para validar zonas de origen/destino, calcular distancias o rutas óptimas, y gestionar la disponibilidad de servicios por zona.

VehiculoService.java: Contiene la lógica de negocio para registrar vehículos, actualizar su ubicación en tiempo real (integración GPS), y gestionar su disponibilidad.

BodegaService.java: Contiene la lógica de negocio para registrar bodegas, gestionar el inventario dentro de ellas, y registrar movimientos de mercancía.

util/

Provee clases auxiliares para funcionalidades transversales.

DateUtil.java: Métodos estáticos para el formateo, parsing y manipulación de fechas y horas.

ResponseUtil.java: Utilidades para construir respuestas API estandarizadas, incluyendo mensajes de éxito, error y estructuras de datos comunes.

ValidationUtil.java: Métodos auxiliares para validaciones de datos más allá de las anotaciones de modelo, por ejemplo, validaciones de lógica de negocio compleja.

GeolocalizacionUtil.java: Utilidades para el procesamiento de datos de geolocalización, como cálculo de distancias entre puntos GPS, validación de coordenadas, etc.

5. Tecnologías Utilizadas

El desarrollo del sistema se ha basado en las siguientes tecnologías y *frameworks*:

Lenguaje de Programación: Java (versión 17 o superior)

Framework: Spring Boot (versión 3.x)

Spring Web (para desarrollo de APIs RESTful)

Spring Security (para autenticación y autorización)

Base de Datos: PostgreSQL, Oracle

Acceso a Datos: JDBC con una fábrica de conexiones personalizada y archivo de propiedades

Herramientas de Testing:

JUnit 5: Framework para pruebas unitarias.

Mockito: Framework para creación de *mocks* y simulación de dependencias en pruebas unitarias.

Spring Boot Test: Para pruebas de integración.

Documentación de API: Swagger UI / OpenAPI

Gestión de Dependencias: Maven

Control de Versiones: Git

6. Despliegue

El sistema está empaquetado como un archivo JAR ejecutable, lo que facilita su despliegue en diversos entornos.

Requisitos Previos:

1. Java Runtime Environment (JRE) 17 o superior.
2. Acceso a la base de datos configurada.

Pasos de Despliegue:

3. Compilar el proyecto utilizando Maven: `mvn clean install`
4. El archivo JAR ejecutable se encontrará en la carpeta `target/` (ej. `gestion_logistica-0.0.1-SNAPSHOT.jar`).
5. Ejecutar la aplicación desde la línea de comandos: `java -jar gestion_logistica-0.0.1-SNAPSHOT.jar`

Configuración de Entorno: Las propiedades de conexión a la base de datos y otros parámetros configurables pueden ser sobrescritos mediante variables de entorno o archivos `application.properties/application.yml` externos al JAR.

7. Manejo de Errores

El sistema implementa una estrategia centralizada de manejo de errores para proporcionar respuestas consistentes y útiles a los clientes de la API.

- Excepciones Personalizadas: Se definen excepciones personalizadas para errores específicos de negocio (ej. `RecursoNoEncontradoException`, `ValidacionException`).
- Manejadores Globales de Excepciones: Utiliza `@ControllerAdvice` y `@ExceptionHandler` de Spring para interceptar excepciones en toda la aplicación y transformarlas en respuestas HTTP estandarizadas (códigos de estado y cuerpos JSON descriptivos).

- Validación de Entrada: Se utilizan anotaciones de validación (**@Valid**, **@NotNull**, **@Size**) en los objetos de transporte de datos (DTOs) y *endpoints* para validar la entrada antes de que llegue a la lógica de negocio, retornando errores **400 Bad Request** con detalles específicos.
- Registro de Errores: Todos los errores significativos se registran en los *logs* de la aplicación para facilitar el diagnóstico y la depuración.

PROCESOS DE MEJORA

Agregar reglas personalizables según el tipo de cliente:

Objetivo: Adaptar la logística a las características particulares del cliente y su entorno.

Ejemplos:

- Si la zona de entrega de la mercancía es rural, entonces aumentar la fecha de entrega a dos días.
- Si el cliente paga por prioridad, entonces enviará la mercancía de ese cliente en las siguientes horas.

Ventajas:

- Aumento en la precisión de las promesas de entrega.
- Mejora en la satisfacción del cliente mediante un servicio personalizado.
- Adaptabilidad ante condiciones logísticas complejas.

Seguimiento en tiempo real de la mercancía:

Ejemplos:

- Que cuando la mercancía esté en camino, ya sea avión, camión o barco, el usuario pueda ver en el mapa la ruta de su mercancía.
- Que cuando se esté repartiendo la mercancía, el sistema organice las entregas dependiendo la dirección de cada usuario, de tal forma que se pueda disminuir los tiempos de entrega.

Ventajas:

- Mayor confianza y tranquilidad del cliente.
- Reducción en los tiempos de entrega mediante **ruteo inteligente**.
- Mejora en la eficiencia operativa de los transportistas.
-

Arquitectura basada en microservicios:

Objetivo: Facilitar el mantenimiento, escalabilidad y evolución del sistema.

Características:

- Cada módulo (seguimiento, reglas de entrega, optimización de rutas) es **independiente y desacoplado**.
- Se pueden actualizar o escalar funcionalidades **sin afectar el sistema completo**.

Ventajas:

- Mayor resiliencia ante errores.
- Posibilidad de escalar vertical u horizontalmente según la demanda.
- Desarrollo más ágil e independiente por cada equipo técnico.

-

Modo simulación:

Objetivo: Permitir pruebas seguras de rutas y decisiones logísticas antes de llevarlas a producción.

Funcionalidades:

- Simulación de nuevas rutas de entrega para **diversificación y reducción de riesgo**.
- Evaluación de escenarios alternativos en función de tiempo, tráfico, distancia y costos.

Ventajas:

- Anticipación a posibles cuellos de botella.
- Ahorro en costos por pruebas físicas innecesarias.
- Mejora continua basada en datos simulados realistas.

Enlace al proyecto desplegado

<https://gestionlogistica-63b83.web.app>

Conclusiones

La precisión en la geolocalización es crucial para garantizar que los vehículos de mensajería sigan las rutas correctas, evitando desvíos que puedan resultar en extravíos de mercancía o retrasos. La integración de tecnologías avanzadas como GPS de alta precisión y algoritmos de optimización en tiempo real es esencial para lograr una gestión eficiente y confiable de las rutas.

La optimización dinámica de rutas, basada en condiciones del tráfico y el clima, también es fundamental para garantizar entregas rápidas y seguras.

Los clientes finales son esenciales en el proceso, pero su poder sobre las decisiones operativas es limitado. Sin embargo, mantenerlos informados sobre el estado de sus entregas (por ejemplo, con notificaciones automáticas) aumentará la satisfacción y la confianza en el servicio.

Es importante no solo cumplir con sus expectativas en cuanto a tiempo y seguridad, sino también proporcionarles transparencia en el proceso de entrega. Esto ayuda a fortalecer la relación con el cliente y reduce la posibilidad de quejas o insatisfacción.

El desarrollo de un sistema de geolocalización para carros de mensajería es una solución innovadora que mejora la eficiencia, la precisión en las entregas y la satisfacción del cliente. Sin embargo, cada uno con sus propios intereses y niveles de influencia. Un enfoque equilibrado que considere tanto la optimización tecnológica como las necesidades y expectativas de las partes interesadas garantizará que el proyecto no solo sea funcional, sino también rentable y sostenible en el tiempo.

Ciclo de vida del desarrollo

1. Análisis de requisitos

- Se identificaron las necesidades específicas del sistema, como el manejo de inventarios, seguimiento de pedidos, control de transporte y generación de reportes.
- La comunicación con los usuarios clave (como operadores logísticos y administradores) fue fundamental para definir funcionalidades reales y evitar suposiciones erróneas.
- Se priorizaron requisitos funcionales y no funcionales (seguridad, rendimiento, escalabilidad).

2. Diseño del sistema

- Se estructuró el sistema en MVC el backend y el frontend , lo cual favorece la mantenibilidad.
- Se diseñaron los modelos de datos (UML, ERD) considerando relaciones entre entidades como clientes, transportistas, pedidos y almacenes.
- Se seleccionaron tecnologías adecuadas como Spring Boot (backend), Angular (frontend) y bases de datos relacionales postgresql o tan sensible y oracle para manejo de información sensible.

3. Implementación

- La separación de responsabilidades permitió una codificación más limpia y testeable.
- Se utilizaron buenas prácticas como control de versiones (Git), pruebas unitarias y principios

SOLID.

- El desarrollo iterativo facilitó validar funcionalidades con entregas parciales.

4. Pruebas

- Se realizaron pruebas unitarias y de integración las cuales quedan pendientes por fases posteriores del proyecto con el fin de asegurar la lógica de negocio.
- Se identificaron errores en flujos críticos como el registro de envíos o la asignación de transportistas, que fueron corregidos antes de producción.
- Las pruebas con usuarios reales ayudaron a detectar errores de usabilidad y validación de datos.

5. Despliegue

- Se prepararon contenedores Docker y scripts de despliegue para asegurar consistencia entre ambientes.
- Se usa Openshift y rosa para el uso de CI/CD para realizar un mejor manejo de despliegue de la aplicación
- Se configuró un entorno de testing para validar el sistema antes del paso a producción.
- Se documentó el proceso de despliegue y los requisitos de infraestructura.

6. Mantenimiento

- Se corrigieron errores reportados tras la entrega inicial y se dejaron listos en el documento para corrección en fase posteriores.
- Se atendieron solicitudes de mejora basadas en la retroalimentación de usuarios.
- Se evaluó la necesidad de refactorizar partes del sistema con bajo rendimiento o alta complejidad.

Prototipo de baja fidelidad:

<https://balsamiq.cloud/sbpx3qs/panyh3k>

Prototipo de alta fidelidad:

<https://www.figma.com/design/gDEns4nNzo4RawTKrfoESI/TCCGestionLogistica-team-library?node-id=0-1&t=NaOoP28WQ26apMYV-1>

Url video explicativo

<https://youtu.be/6H1Zh3YhCt8>

url que contiene las fuentes del proyecto en su fase creación de prototipos iniciales

<https://github.com/guiovannycaro/gestionLogistica.git>

url directa al tablero de jira

<https://guiovannycaro.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog>

Referencias

<https://excelparatodos.com/matriz-de-riesgos/>

<https://blog.comparasoftware.com/matriz-de-riesgos-que-es/>

<https://www.piranirisk.com/es/blog/matriz-de-riesgos-que-es-ejemplos-y-como-crearla-facil>

<https://medium.com/@requeridosblog/requerimientos-funcionales-y-no-funcionales-ejemplos-y-tips-aa31cb59b22a>

<https://visuresolutions.com/es/gu%C3%ADa-de-trazabilidad-de-gesti%C3%B3n-de-requisitos/requisitos-funcionales-vs-no-funcionales/>

<https://www.lucidchart.com/blog/es/como-elaborar-diagramas-de-arquitectura>

<https://aws.amazon.com/es/what-is/architecture-diagramming/>

<https://www.yarquitectura.com/diagrama-de-arquitectura-de-software/>

<https://visuresolutions.com/es/gu%C3%ADa-de-trazabilidad-de-gesti%C3%B3n-de-requisitos/requisitos-funcionales-vs-no-funcionales/>

<https://single-spa.js.org/docs/ecosystem-angular/>

<https://es.single-spa.js.org/docs/examples/>

<https://refactoring.guru/es/design-patterns>

Pressman, Roger S., (2021) Ingeniería de software. McGraw-Hill Interamericana. Capítulo 24, 25, 26 página de la 490 a 548

Gual Ortí, J. (2016). Fundamentos del modelado y prototipado virtual en el diseño de productos.. D - Universitat Jaume I. Servei de Comunicació i Publicacions. Capítulo 2 y 3 páginas 3 a la 5.

Suárez, E. C. (2017). Prototipo, Contexto e Ingeniería del Software. Estudios de Postgrado, en Sistemas de Información, 2-20.

Actividad 4 - Diseño del prototipo.

JUAN SEBASTIAN MARTINEZ HIGUITA

CRISTIAN DAVID RESTREPO ORTIZ

GIOVANNY ANATOLI CARO DAZA

JIMMY ALEXANDER MENJURA CALVO

FACULTAD DE INGENIERÍAS, UNIVERSIDAD IBEROAMERICANA

PROYECTO DE SOFTWARE

DTE. TATIANA CABRERA

Actividad 4 - Diseño del prototipo.

JUAN SEBASTIAN MARTINEZ HIGUITA

CRISTIAN DAVID RESTREPO ORTIZ

GIOVANNY ANATOLI CARO DAZA

JIMMY ALEXANDER MENJURA CALVO

FACULTAD DE INGENIERÍAS, UNIVERSIDAD IBEROAMERICANA

PROYECTO DE SOFTWARE

DTE. TATIANA CABRERA

04 DE MAYO DE 2025

Tabla Contenido

Introducción	1
Abstract	2
Necesidad	3
Alcance	4
Restricciones	5
Criterios de Aceptación	6
Descripción del Problema	7
Objetivos	8
Objetivo General	8.1
Objetivo Especifico	8.2
Fase de Diseño	9
1. RQF-RQNF	9.1
2. HU (enlace tablero de la metodología ágil)	9.2
Modelamiento	10
Diagrama de clases-diagrama de objetos- diagrama de componentes	10.1
Diagrama de casos de uso - diagrama de secuencias -diagrama de estados	10.2
nociones básicas de diseño: prototipos de baja fidelidad	10.3
-prototipos de alta fidelidad(enlace de herramienta utilizada) - Mapa de navegación.	10.4
Fase de Desarrollo	11

Codificar min. 2 funcionalidades según la necesidad planteada y los RQF	11.1
Repositorio enlace de GitHub	11.2
Fase de Pruebas	12
Pruebas unitarias	12.1
Enlace video de como funciona el sistema	12.2
procesos futuros.	13
Conclusiones	14
Referencias	15

INTRODUCCIÓN

En la actualidad, la gestión eficiente de la cadena de suministro es un factor clave para el éxito de cualquier empresa que maneje mercancías a nivel nacional e internacional. Con el crecimiento del comercio global, la necesidad de contar con un sistema confiable y preciso para el monitoreo y control de los productos en tránsito se ha vuelto fundamental.

La empresa XXX, que se encarga del transporte de mercancías a diferentes partes del mundo por vía terrestre, marítima o aérea, ha venido presentando diversos inconvenientes en el proceso de transporte. Entre los problemas más comunes se encuentran los atrasos en la entrega de mercancías debido a fallas logísticas, errores en la coordinación de envíos y demoras en el despacho. Además, se han reportado daños en la mercadería transportada, lo que genera insatisfacción en los clientes y pérdidas económicas para la empresa.

Estos problemas han afectado la reputación de la compañía y han generado la necesidad de implementar una solución tecnológica que permita mejorar la gestión del transporte. Por esta razón, la empresa ha decidido desarrollar un sistema que centralice y optimice el control de los procesos de envío, seguimiento y entrega de mercancías. Este sistema permitirá a la empresa monitorear en tiempo real el estado de cada envío, prevenir retrasos mediante la optimización de rutas y mejorar la seguridad de los productos transportados.

Además, el sistema proporcionará herramientas para gestionar la comunicación con los clientes, notificándose sobre el estado de sus envíos y permitiéndoles realizar un seguimiento detallado de sus pedidos. Con estas mejoras, la empresa busca garantizar una mayor satisfacción del cliente, optimizar sus operaciones logísticas y mantener su competitividad en el mercado global del transporte de mercancías.

Este proyecto tiene como objetivo el desarrollo de un sistema de control de mercancía que permita a los dueños de los productos conocer en todo momento la ubicación exacta de sus envíos, en qué bodega se encuentran y por qué medios de transporte están siendo trasladados, ya sea por barco, avión o camión.

El sistema propuesto proporcionará información en tiempo real, mejorando la toma de decisiones y reduciendo los riesgos asociados a la pérdida, retrasos o daños en la mercadería. Adicionalmente,

permitirá a los usuarios optimizar la logística de sus operaciones, minimizando costos y mejorando la eficiencia en la distribución.

Mediante el uso de tecnologías avanzadas como IoT, bases de datos seguras y geolocalización, el sistema será capaz de recopilar, procesar y mostrar información relevante para cada envío. Esto no solo permitirá un mejor seguimiento de la mercadería, sino también una mayor transparencia y confianza en la relación entre proveedores, transportistas y clientes.

Abstract

En el contexto actual del comercio global, la eficiencia en la gestión de la cadena de suministro representa un factor crítico para el éxito de las empresas dedicadas al transporte de mercancías. La empresa XXX, especializada en el traslado de productos por vía terrestre, marítima y aérea, ha enfrentado múltiples desafíos operativos, tales como retrasos en las entregas, errores de coordinación logística, y daños en la mercadería. Estas dificultades han impactado negativamente en la satisfacción del cliente y la reputación de la compañía, generando la necesidad de adoptar una solución tecnológica integral.

Este proyecto propone el desarrollo de un sistema inteligente de control y monitoreo de mercancías, diseñado para optimizar el seguimiento en tiempo real, mejorar la seguridad de los productos en tránsito y fortalecer la comunicación con los clientes. El sistema utilizará tecnologías como geolocalización (GPS), Internet de las Cosas (IoT) y bases de datos seguras para ofrecer visibilidad total sobre la ubicación, medio de transporte y almacenamiento de cada envío. Además, incluirá funcionalidades como alertas por incidentes, generación de reportes y protección de datos mediante cifrado.

Con esta iniciativa, se busca minimizar los riesgos logísticos, reducir costos operativos, mejorar la toma de decisiones y reforzar la competitividad de la empresa en el mercado internacional de transporte de mercancías.

Necesidad

Las empresas de transporte enfrentan constantemente el desafío de mantener el control sobre el envío y la recepción de mercancías. En particular, la empresa ***** dedicada al transporte marítimo, terrestre y aéreo ha experimentado problemas recurrentes relacionados con el extravío de mercancías durante el trayecto. Esto genera pérdidas económicas, afectación a la reputación y problemas de gestión.

Para abordar esta problemática, se ha decidido implementar un software especializado que permita registrar y monitorear el envío de mercancías, garantizando el control desde el punto de partida hasta la llegada a destino. Además, se pretende utilizar sistemas de geolocalización (GPS) para rastrear en tiempo real el trayecto de los vehículos, controlar el tiempo estimado de llegada, el estado de la mercancía, el vehículo que realiza el traslado y la bodega de recepción.

Alcance del Proyecto

El sistema de control de mercancía abarca los siguientes aspectos:

- Monitoreo en tiempo real: Los dueños de la mercancía podrán visualizar la ubicación de sus productos en cualquier momento.
- Registro de almacenamiento: Identificación de la bodega en la que se encuentra la mercancía y su historial de movimientos.
- Seguimiento de transporte: Información detallada sobre los medios utilizados para el traslado (barco, avión o camión).
- Seguridad de datos: Protección de la información mediante cifrado y protocolos de acceso seguro.
- Alertas y notificaciones: Envío de alertas en caso de retrasos, cambios de ruta o incidentes con la mercancía.

- Generación de reportes: Creación de informes sobre el estado y movimiento de la mercancía para mejorar la gestión logística.
- Rastreo satelital:
-

Restricciones

- El sistema dependerá de la disponibilidad de internet para la actualización en tiempo real.
- No incluye gestión de pagos o facturación de envíos.
- La precisión de la geolocalización estará sujeta a la cobertura de los dispositivos GPS utilizados.

Criterios de Aceptación del Proyecto

- Permitir la consulta en tiempo real de la ubicación de la mercancía.
- Garantizar la seguridad de la información con autenticación de usuarios.
- Proporcionar reportes detallados de los movimientos de la mercancía.
- Ser accesible desde dispositivos móviles y computadoras con conexión a internet.

Descripción del Problema

La empresa XXX, que se encarga del transporte de mercancías a diferentes partes del mundo por vía terrestre, marítima o aérea, ha venido presentando diversos inconvenientes en el proceso de transporte. Entre los problemas más comunes se encuentran los atrasos en la entrega de mercancías debido a fallas logísticas, errores en la coordinación de envíos y demoras en el despacho. Además, se han reportado daños en la mercadería transportada, lo que genera insatisfacción en los clientes y pérdidas económicas para la empresa.

Estos problemas han afectado la reputación de la compañía y han generado la necesidad de implementar una solución tecnológica que permita mejorar la gestión del transporte. Por esta razón, la empresa ha decidido desarrollar un sistema que centralice y optimice el control de los procesos de envío, seguimiento y entrega de mercancías. Este sistema permitirá a la empresa monitorear en tiempo real el estado de cada envío, prevenir retrasos mediante la optimización de rutas y mejorar la seguridad de los productos transportados.

Además, el sistema proporcionará herramientas para gestionar la comunicación con los clientes, notificándose sobre el estado de sus envíos y permitiéndoles realizar un seguimiento detallado de sus pedidos. Con estas mejoras, la empresa busca garantizar una mayor satisfacción del cliente, optimizar sus operaciones logísticas y mantener su competitividad en el mercado global del transporte de mercancías.

Objetivos

Objetivo General: Desarrollar un software de gestión y monitoreo para la empresa *****, que permita el control eficiente de los envíos y la recepción de mercancías, optimizando la trazabilidad y reduciendo el riesgo de pérdida o daño durante el trayecto.

Objetivos Específicos:

1. Implementar un sistema de registro para el control de los envíos y las recepciones en las bodegas.
2. Integrar un módulo de geolocalización (GPS) para monitorear en tiempo real el recorrido de los vehículos.
3. Desarrollar un sistema de control que registre el tiempo de desplazamiento y el estado de la mercancía desde el punto de partida hasta el destino.
4. Permitir el almacenamiento de información sobre el vehículo utilizado y la bodega de recepción, asegurando la trazabilidad completa del proceso.

Fase de Diseño

Requisitos Funcionales y No Funcionales (RQF - RQNF)

Requisitos Funcionales (RQF)

- **RQF1:** El sistema debe permitir registrar envíos logísticos.
- **RQF2:** El sistema debe permitir consultar el estado de un envío.
- **RQF3:** El sistema debe permitir gestionar inventarios por bodega.
- **RQF4:** El administrador puede asignar rutas de entrega.

Requisitos No Funcionales (RQNF)

- **RQNF1:** La aplicación debe estar disponible 99% del tiempo.
- **RQNF2:** Las respuestas del backend no deben superar 2 segundos.
- **RQNF3:** El sistema debe ser escalable horizontalmente.
- **RQNF4:** Cumplir con OWASP para evitar vulnerabilidades comunes.

Historias de Usuario (HU)

Para el desarrollo de esta plataforma se va a usar jira como herramienta de control de versiones acontinuacion se muestra un resumen de las historias de usuario mas usadas dentro del sistema

Historias de Usuario por Módulo

Autenticación y Gestión de Usuarios

HU-001: Como administrador, quiero poder registrar nuevos usuarios en el sistema para gestionar su acceso.

HU-002: Como usuario, quiero iniciar sesión en el sistema con mi correo y contraseña para acceder a mis funciones.

HU-003: Como administrador, quiero asignar roles y permisos a los usuarios para restringir el acceso a ciertas funciones.

HU-004: Como usuario, quiero poder recuperar mi contraseña en caso de olvido para restablecer mi acceso.

Gestión de Transportistas y Vehículos

HU-005: Como administrador, quiero registrar transportistas en el sistema para asignarles envíos.

HU-006: Como operador logístico, quiero consultar la lista de transportistas disponibles para optimizar asignaciones.

HU-007: Como transportista, quiero recibir notificaciones sobre los envíos asignados para planificar mis rutas.

HU-008: Como administrador, quiero gestionar la información de los vehículos para mantener un registro actualizado.

HU-009: Como transportista quiero gestionar el envío de información al sistema con el fin de reportar novedades con el fin de solucionar algún posible retraso.

Gestión de Mercancías y Almacenes

HU-0010: Como cliente, quiero registrar un nuevo envío en el sistema para solicitar su transporte.

HU-0011: Como operador logístico, quiero asignar mercancías a transportistas para gestionar su distribución.

HU-0012: Como administrador, quiero consultar el estado de todas las mercancías para monitorear su ubicación.

HU-0013: Como cliente, quiero recibir una notificación cuando mi mercancía haya sido entregada.

Seguimiento y Rastreo de Envíos

HU-0014: Como cliente, quiero rastrear mi envío en tiempo real para conocer su ubicación actual.

HU-0015: Como operador logístico, quiero recibir alertas de retrasos para tomar decisiones correctivas.

HU-0016: Como transportista, quiero actualizar el estado del envío en el sistema para mantener informados a los clientes.

HU-0017: Como cliente, quiero recibir notificaciones de cambios en el estado de mi envío para estar al tanto.

Módulo de Solicitudes

HU-0018: Como cliente, quiero enviar solicitudes de transporte de mercancías para gestionar mis envíos.

HU-019: Como operador logístico, quiero revisar y aprobar solicitudes de transporte para garantizar disponibilidad.

HU-020: Como cliente, quiero cancelar una solicitud antes de que sea confirmada para evitar costos innecesarios.

HU-021: Como transportista, quiero recibir notificaciones sobre nuevas solicitudes asignadas para planificar mis entregas

Notificaciones y Reportes

HU-022: Como administrador, quiero generar reportes sobre envíos completados, retrasos y problemas para analizar el rendimiento.

HU-023: Como cliente, quiero recibir un correo o mensaje cuando mi mercancía sea entregada para confirmar la recepción.

HU-024: Como operador logístico, quiero recibir alertas de posibles problemas en la entrega para tomar medidas.

HU-025: Como transportista, quiero enviar alertas sobre accidentes o problemas en ruta para notificar retrasos.

Modelamiento

Diagrama de arquitectura del sistema

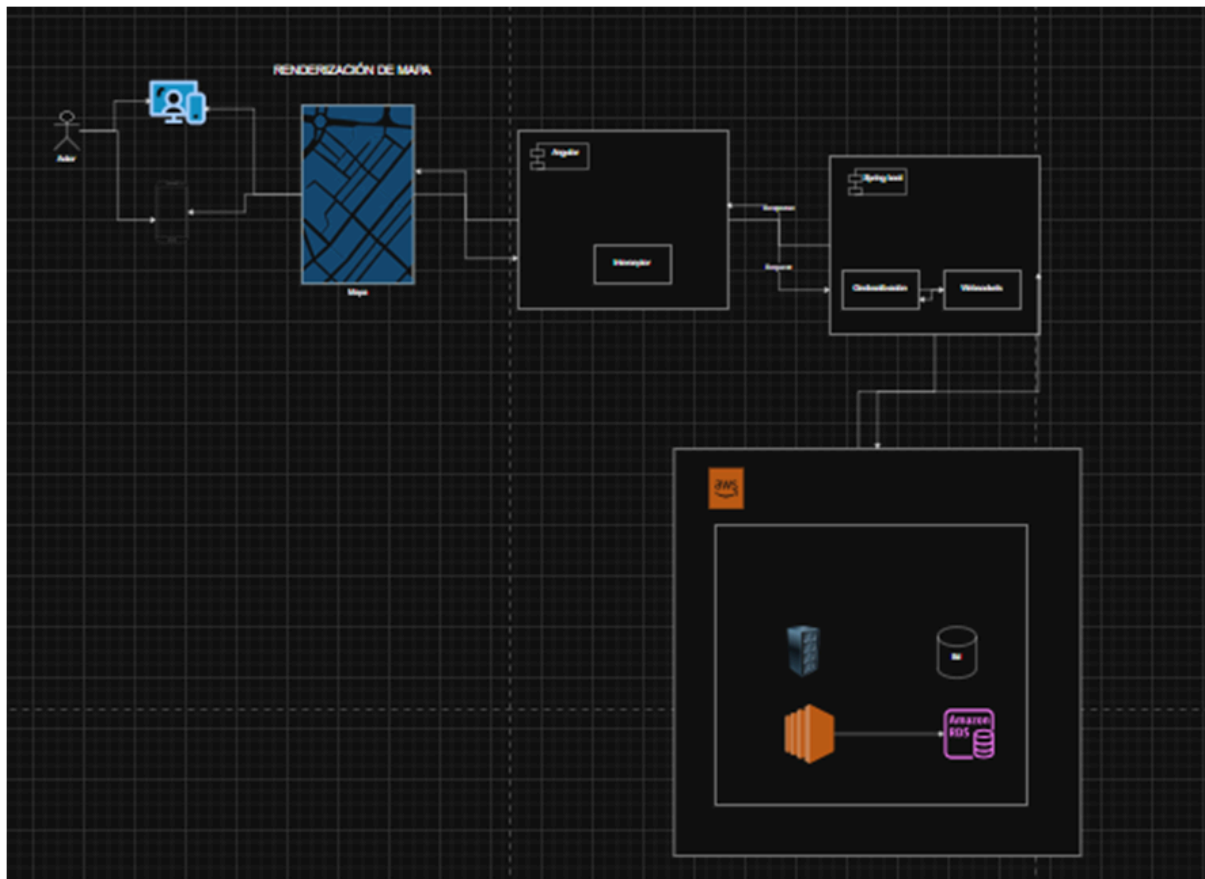


Diagrama de flujo

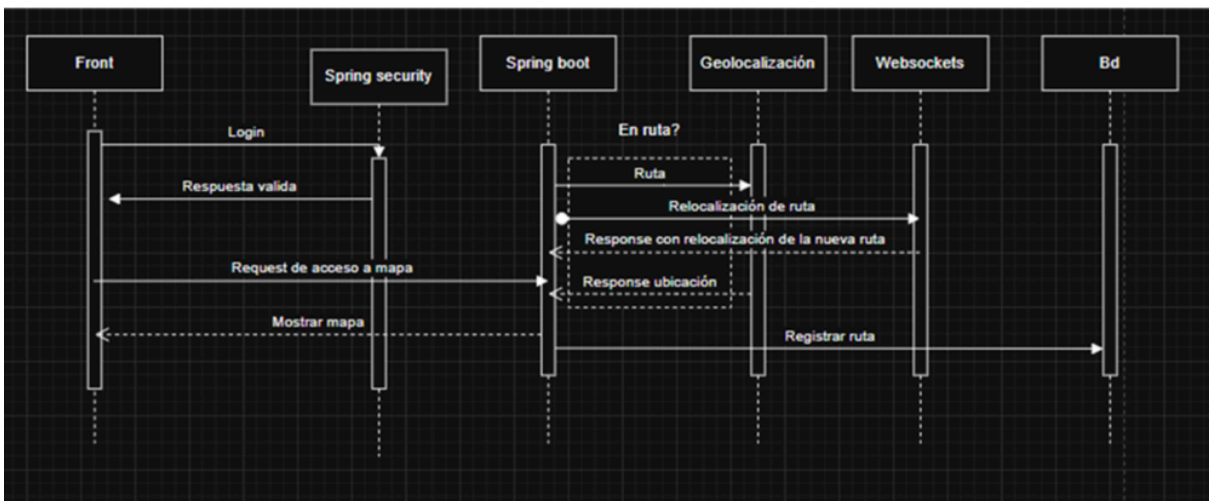
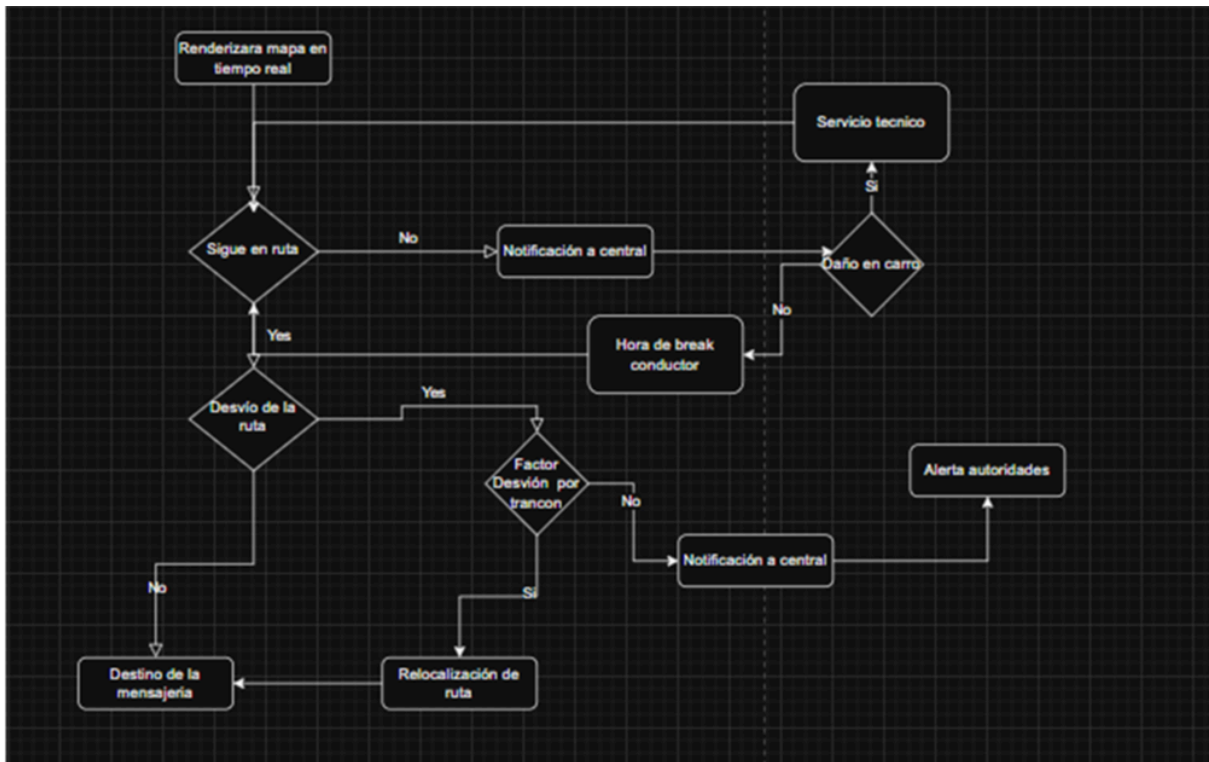
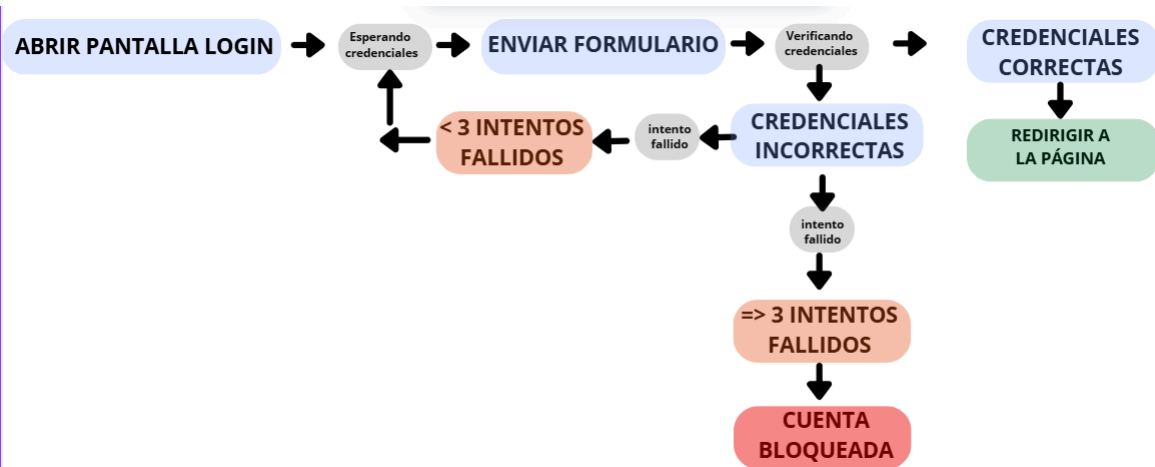
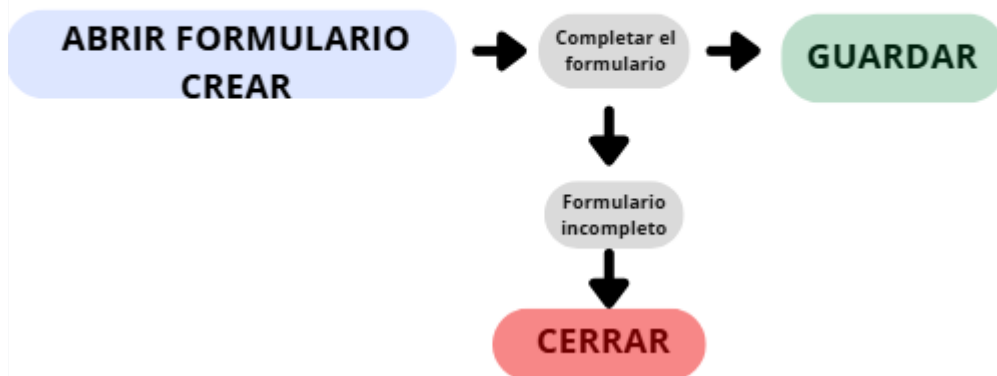


DIAGRAMA DE ESTADOS DEL LOGIN



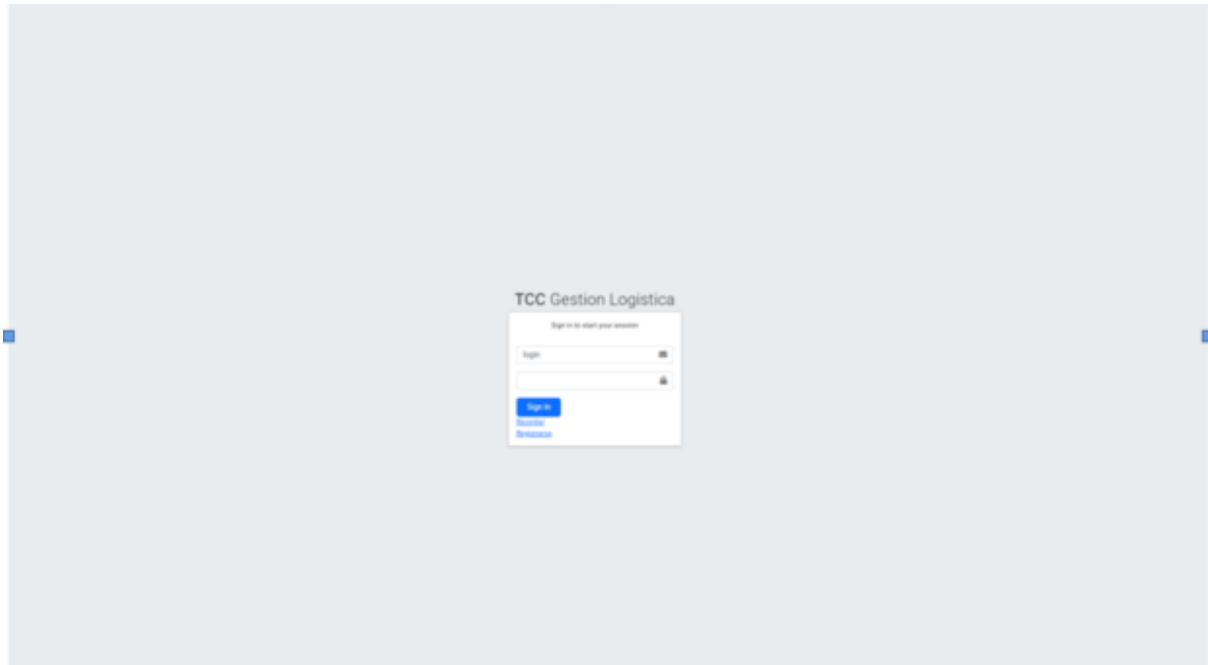
CREAR ENVÍO

CREAR ENVÍO

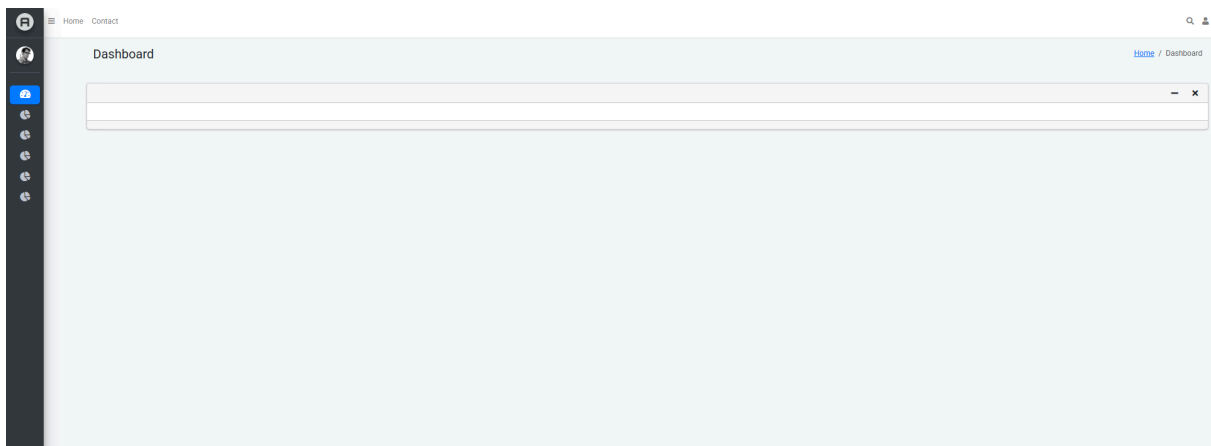


ESTADOS DE ENVÍOS

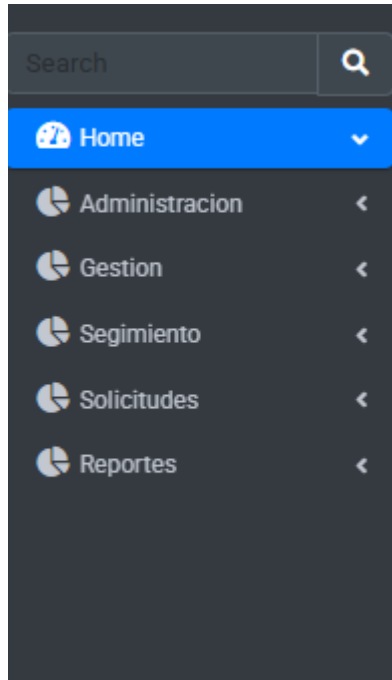




Dashboard Prototipo

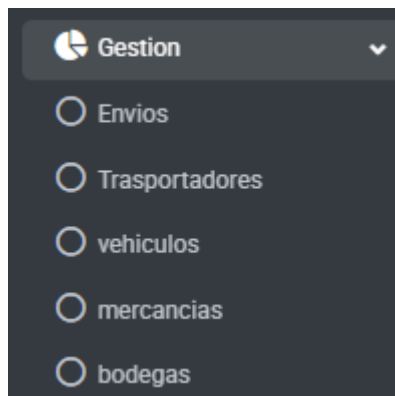


Menu



- **RQF2:** El sistema debe permitir consultar el estado de un envio.

Menu Envios



seccion envios

ID	Codigo Cliente	Telefono	Codigo Bodega	Fecha Envio	Fecha Entrega	Ciudad Origen	Ciudad Destino	Estado	Accion
101	2	3160405672	2	12/02/2025	15/03/2025	1	1	1	[Editar] [Eliminar]
101	2	3160405672	2	12/02/2025	15/03/2025	1	1	1	[Editar] [Eliminar]
12	2	3160405672	2	12/02/2025	15/03/2025	1	1	2	[Editar] [Eliminar]
104	2	3160405672	1	12/02/2025	15/03/2025	1	1	1	[Editar] [Eliminar]

nuevo envio RQF1: El sistema debe permitir registrar envios logísticos.

REFERENCIA

 Cliente
 Telefono
 Bodega
 Fecha Entrega
 Ciudad Destino
 Fecha Envio
 Ciudad Origen
 Estado

Cargar Mercancias
 Mercancias
 Cantidad
 Descuento
 Descuento
 + Agregar Mercancia

Id	Producto	Cantidad	Precio	Total	Accion
----	----------	----------	--------	-------	--------

lo mismo en editar envío

Para Acceder a esta sección en el menú seleccionar el módulo gestión y a continuación damos en el enlace a envíos

Caso Uso 1 :Permitir la autenticación de los usuarios.

El sistema debe permitir el ingreso del nombre y password del usuario para realizar las diferentes funciones que tendrá cada uno.

ENTRADAS: Cédula, Nombres, Apellidos, Contraseña, Tipo de Usuario (Administrador, Operador, Transportista, cliente).

PROCESOS El sistema pedirá la correspondiente identificación como administrador,. Nos ubicamos

en la parte de Administración del menú principal y escogemos Usuario El sistema pedirá los correspondientes datos del nuevo usuario luego verificará que no haya espacios en blanco, en el caso de ningún error guardará los datos del nuevo usuario.

SALIDAS Las salidas van dirigidas a el Superadministrador o Administrador Del sistema . Mensaje de error en el caso de no haber llenado algún campo. Mensaje de error en el caso de ingresar un numero de cedula ya existente en la base de datos Mensaje de error en casos de ingresar incorrectamente los datos es decir que el formato de los datos sea incorrecto

Caso de Uso 1: Registrar Envío

OBJETIVO: Permitir que el sistema registre un nuevo envío ingresado por un cliente, operador o administrador.

ENTRADAS:

documento del cliente

teléfono de contacto del cliente

bodega de envío de la mercancía

fecha de entrega

fecha envio

ciudad de origen del envío

ciudad destino del envío

estado del envío

la referencia (descripción Tipo de envío normal, express, etc.)

PROCESOS:

Desde el menú principal de la opción gestión, el usuario selecciona la opción "Envío". El sistema presenta un formulario modal con los campos requeridos.

El usuario ingresa los datos del envío. El sistema valida:

- Que no haya campos obligatorios vacíos
- Que las direcciones sean válidas

Si no hay errores, el sistema guarda el envío, asigna un número de guía único y establece su estado inicial como "Pendiente".

SALIDAS:

- Mensaje de confirmación con el número de guía
- Mensaje de error si falta algún campo obligatorio
- Mensaje de error si los datos no cumplen con el formato requerido (por ejemplo, la dirección esté correctamente digitada)

Caso de Uso 3: Asignar Ruta

OBJETIVO: Permitir al administrador o operador asignar una ruta de entrega a uno o varios envíos registrados.

ENTRADAS:

- Número de guía del o envío
- Ruta seleccionada
- Transportista

PROCESOS:

Desde el menú de administración, se accede a la opción “Asignar Ruta”.

El sistema lista los envíos pendientes de asignación.

El usuario selecciona uno o más envíos y luego la ruta correspondiente.

El sistema valida que los envíos estén en estado “Pendiente” y que la ruta esté activa.

Si no hay errores, el sistema asigna la ruta a los envíos seleccionados.

SALIDAS:

- Confirmación de que la ruta fue asignada correctamente
- Mensaje de error si se intenta asignar una ruta a un envío ya asignado
- Mensaje de error si la ruta no está disponible o si hay campos sin completar

Caso de Uso: Ciclo de Vida de un Envío**OBJETIVO:**

Gestionar los estados por los que pasa un envío desde su creación hasta su entrega final.

ENTRADAS:

- Número de guía o ID del envío
- Evento que dispara el cambio de estado (por ejemplo: asignación de ruta, inicio de viaje, confirmación de entrega)

PROCESOS:

1. El usuario registra un nuevo envío.
Estado inicial: Pendiente
2. Un operador o administrador asigna una ruta al envío.
Estado cambia a: En Ruta
3. Durante el proceso de transporte, se pueden registrar actualizaciones de ubicación o estado intermedio.
4. El transportista confirma la entrega al destinatario.
Estado cambia a: Entregado
5. El sistema guarda el historial de todos los cambios de estado para consulta futura.

Validaciones importantes:

- Solo los envíos en estado pendiente pueden pasar a En Ruta.
- Solo los envíos En Ruta pueden pasar a Entregado.
- El sistema no debe permitir saltos de estado inválidos.

SALIDAS:

Confirmación del cambio de estado

Mensaje de error si el cambio no es válido (por ejemplo, cambiar de "Pendiente" a "Entregado" directamente)

Registro en el historial de eventos del envío

ACTORES:

- Operador logístico
- Transportista

- Administrador del sistema

Secuencias: Interacción Usuario -> Angular -> REST API -> Java -> DB.

Esta representa **el flujo completo** de una solicitud en una aplicación web moderna tipo SPA con backend en Java.

1. Usuario → Angular

El usuario interactúa con la aplicación web en el navegador (por ejemplo, hace clic en un botón para consultar un envío).

Angular, que está corriendo en el navegador, detecta este evento y ejecuta una función para llamar al backend.

2. Angular → REST API

Angular hace una **petición HTTP (GET, POST, etc.)** a un **servidor backend** a través de una **REST API**.

Aquí se pasa la información necesaria (como un número de guía, datos del envío, etc.).

3. REST API → Java

El servidor recibe la petición, el servicio determinado REST de Java se encarga de procesar la solicitud por el controlador correspondiente.

4. Java → DB (Base de Datos)

El backend Java accede a la **base de datos** para obtener o modificar datos.

5. La respuesta sube de nuevo

La base de datos devuelve los datos a Java.

Java prepara una respuesta a través del servicio y se la devuelve al cliente formada como un objeto Json Angular.

Angular recibe los datos y **actualiza la interfaz de la cual salió la petición al servicio del backend** y recargar la página.

Resumen Visual

[Usuario (Navegador)]



[Angular SPA (Frontend)]

↓ (HTTP request)

[REST API - Java (Backend)]



[Base de Datos]



[Java responde]



[Angular muestra los datos]

Estados: Ciclo de vida de un Envío (Pendiente → En Ruta → Entregado).

Ciclo de Vida de un Envío

Un envío pasa por varios **estados operativos** desde que se registra hasta que es entregado. A continuación, los principales:

1. Pendiente

Significado: El envío ha sido registrado en el sistema, pero **aún no ha sido asignado a una ruta** ni ha comenzado el proceso de transporte.

Acciones posibles:

Asignar ruta

Editar datos

Cancelar (si aplica)

2. En Ruta

Significado: El envío ya fue asignado a un transportista o ruta de entrega y **está en tránsito** hacia el destinatario.

Puede incluir sub-estados opcionales como:

"En bodega de origen"

"En vehículo"

"En bodega de destino"

Acciones posibles:

Actualizar ubicación

Marcar como "Reprogramado" si hay un retraso

End points

Estas urls son creadas desde spring boot y consumidas en angular usando http métodos del navegador y métodos que usa para ser consumidas desde el navegador en formato json usando api rest.

Bodegas

Estas urls son para realizar el crud de bodegas create,read,update,delete , cada uno de estos endpoint son esenciales para mostrar las bodegas de nuestro proveedor, para las recogidas de los productos y así mismo llevarlas a nuestras bodegas.

<http://localhost:8090/tcc/gestion/CrudBodega/ListarBodegas>

<http://localhost:8090/tcc/gestion/CrudBodega/crearBodega>

<http://localhost:8090/tcc/gestion/CrudBodega/buscarBodegaById?id=>

<http://localhost:8090/tcc/gestion/CrudBodega/editarBodega>

<http://localhost:8090/tcc/gestion/CrudBodega/eliminarBodega?id=>

Cientes

Estas urls son para realizar el crud de los clientes create,read,update,delete , cada uno de estos endpoint son esenciales para recolectar la información del cliente y tener una pantalla personalizada para estos mismos.

<http://localhost:8090/tcc/gestion/CrudClientes/ListarClientes>

<http://localhost:8090/tcc/gestion/CrudClientes/crearClientes>

<http://localhost:8090/tcc/gestion/CrudClientes/buscarClientesById?id=>

<http://localhost:8090/tcc/gestion/CrudClientes/editarClientes>

<http://localhost:8090/tcc/gestion/CrudClientes/eliminarClientes?id=>

Envíos

Estas urls son para realizar el crud de los envíos. create,read,update,delete , cada uno de estos endpoint son esenciales para mostrar las envios que realizan los clientes, y asi mismo podemos obtener detalladamente usando geolocalización y mapa de donde se encuentra el envío más específicamente para asi mismo poder encontrar cada uno de nuestros envíos adicional podemos observar en que fase se encuentra

<http://localhost:8090/tcc/gestion/CrudDetalleEnvio/ListarDetalleEnvio>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvioTemp/ListarDetalleEnvioTemp>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvioTemp/eliminarDetalleEnvioTemp?id=>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvio/crearDetalleEnvio>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvio/editarDetalleEnvio>

<http://localhost:8090/tcc/gestion/CrudEnvio/buscarEnvioById?id=>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvioTemp/buscarDetalleEnvioTempById?id=>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvioTemp/crearDetalleTempEnvio>

<http://localhost:8090/tcc/gestion/CrudDetalleEnvioTemp/eliminarDetalleEnvioTemp?id=>

Productos

Estas urls son para realizar el crud de Productos create,read,update,delete , cada uno de estos endpoint son esenciales para el registro y obtener los productos que se envían a nuestra plataforma, y asi mismo poder hacer el envío de este mismo

<http://localhost:8090/tcc/gestion/CrudProductos/ListarProductos>

<http://localhost:8090/tcc/gestion/CrudProductos/crearProducto>

<http://localhost:8090/tcc/gestion/CrudProductos/buscarProductosById?id=>

<http://localhost:8090/tcc/gestion/CrudProductos/editarProductos>

<http://localhost:8090/tcc/gestion/CrudProductos/eliminarProductos?id=>

Usuarios

Estas urls son para realizar el crud de usuarios create,read,update,delete , cada uno de estos endpoint son esenciales para realizar la creación y gestion de usuarios por roles, ya que cada uno de estos, tendran la potestad de creación y gestion de productos, asi mismo algunos tendran la potestad de creación de usuarios, y podran observar los reportes generados para los usuarios y tener un mayor control de este

<http://localhost:8090/tcc/gestion/CrudUsuarios/ListarUsuarios>

<http://localhost:8090/tcc/gestion/CrudUsuarios/crearUsuarios>

<http://localhost:8090/tcc/gestion/CrudUsuarios/buscarUsuariosById?id=>

<http://localhost:8090/tcc/gestion/CrudUsuarios/editarUsuarios>

<http://localhost:8090/tcc/gestion/CrudUsuarios/eliminarUsuarios?id=>

Roles del Sistema

Super -Administrador

- **Habilidades:** Gestion clientes, gestión empleados, asignación de roles, Gestión de usuarios, Gestión Operaciones, configuración del sistema, generación de reportes, generación de informes.
- **Actividades:** Administrar el acceso al sistema, supervisar las operaciones y garantizar la seguridad de la información. Generar, analizar y tomar decisiones en función de los reportes que genere el sistema, carga de datos al sistema

Administrador

- **Habilidades:** Gestion clientes, gestión empleados, asignación de roles, Gestión de usuarios, Gestión Operaciones, generación de reportes, generación de informes.
- **Actividades:** Administrar el acceso al sistema, supervisar las operaciones y garantizar la seguridad de la información. Generar, analizar y tomar decisiones en función de los reportes que genere el sistema, carga de datos al sistema

Operador

- **Habilidades:** Manejo de herramientas de rastreo y optimización de rutas.
- **Actividades:** Uso de aplicaciones de rastreo, seguimiento de envíos, seguimientos de las operaciones, asignación de transportistas, carga de datos al sistema, análisis de posibles soluciones a las novedades registradas .

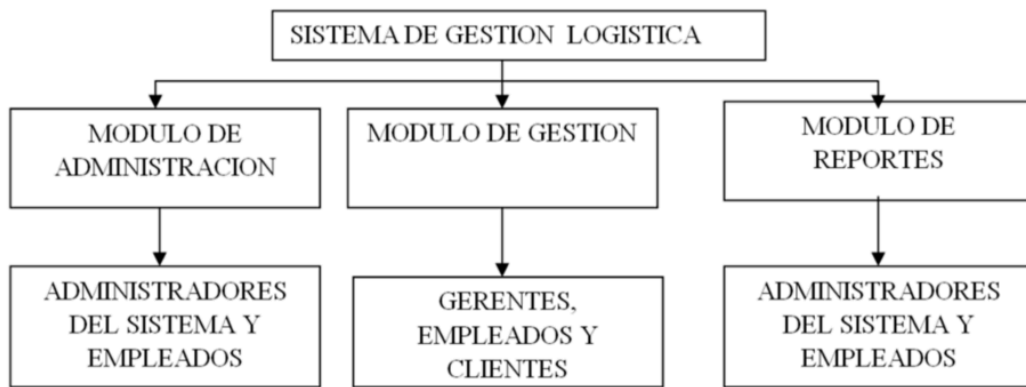
Transportista

- **Habilidades:** Uso de aplicaciones de rastreo y notificación.
- **Actividades:** Transporte de mercancías, actualización de estados de envío, reporte de novedades.

Cliente

- **Habilidades:** Uso básico de la plataforma web/móvil.
- Actividades:** Consulta de envíos, recepción de notificaciones

Diagrama General De Los Módulos del Sistema



Arquitectura del Sistema

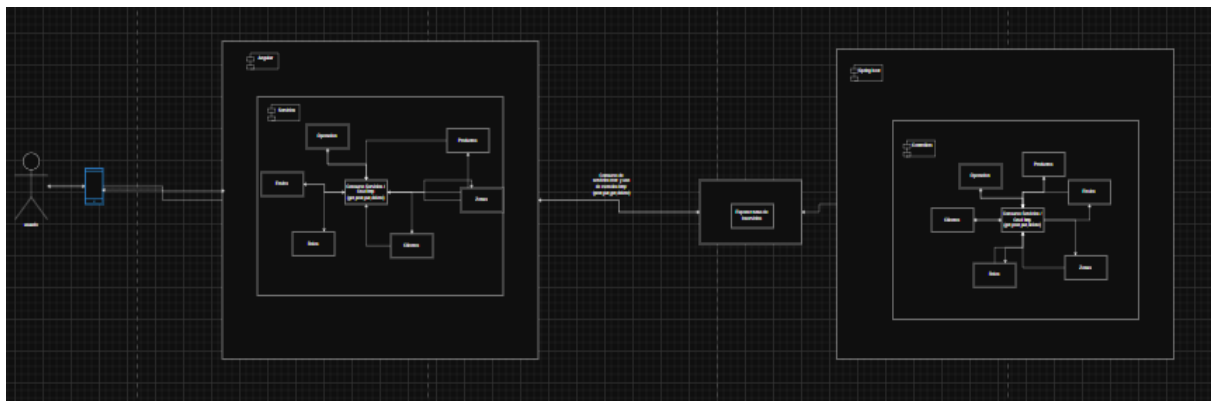


Diagrama de arquitectura Backend

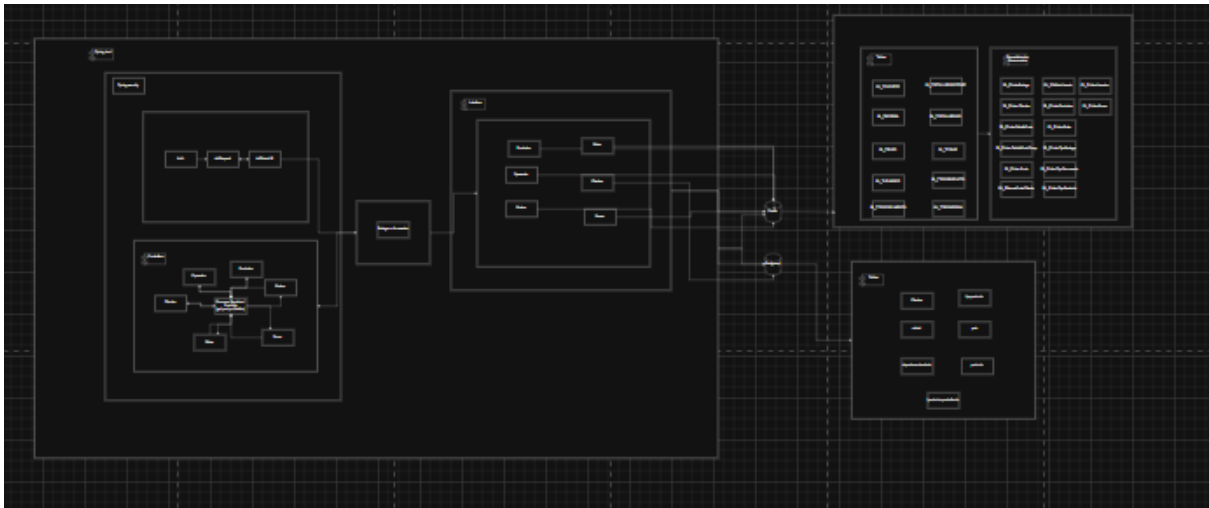
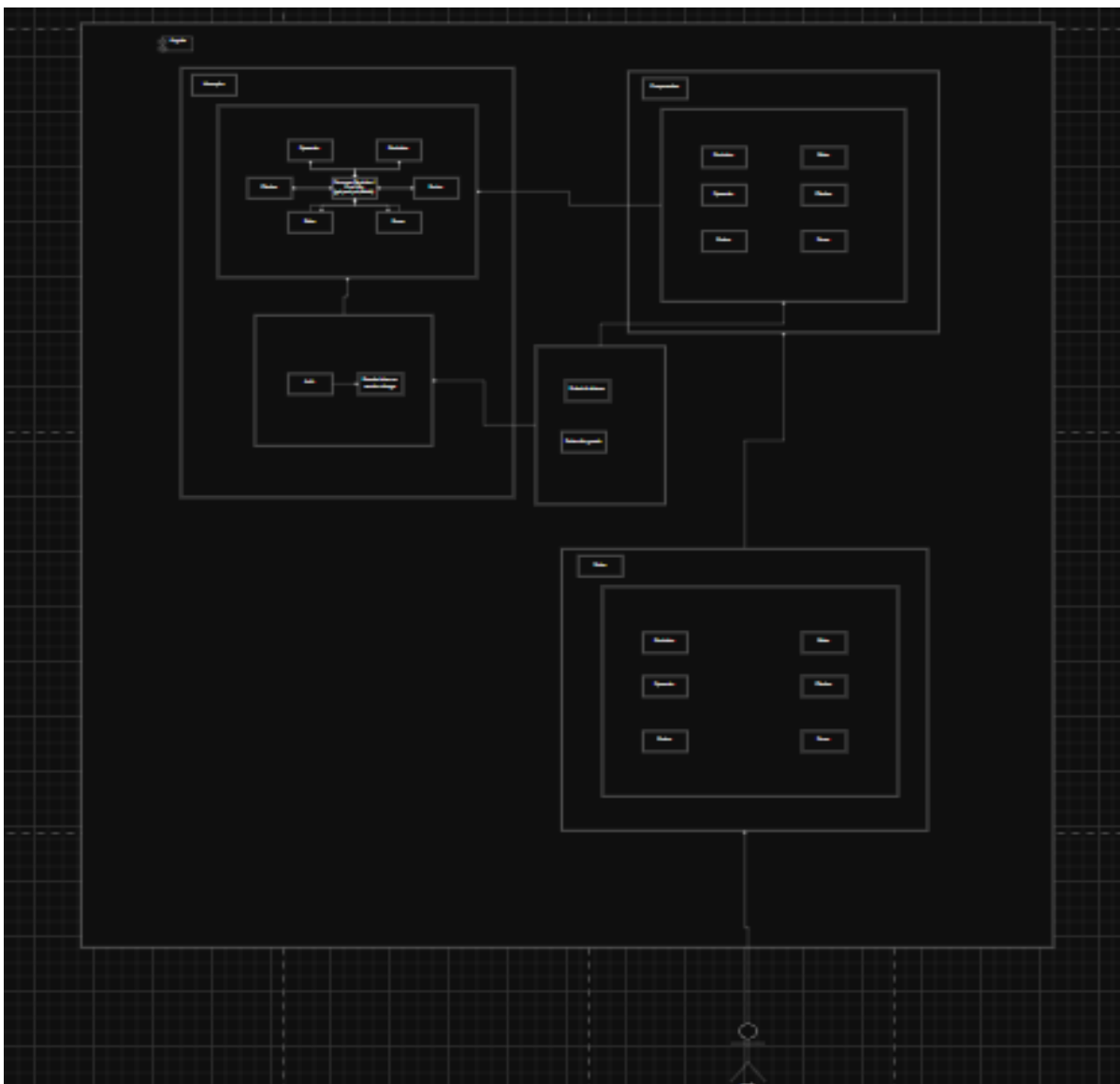
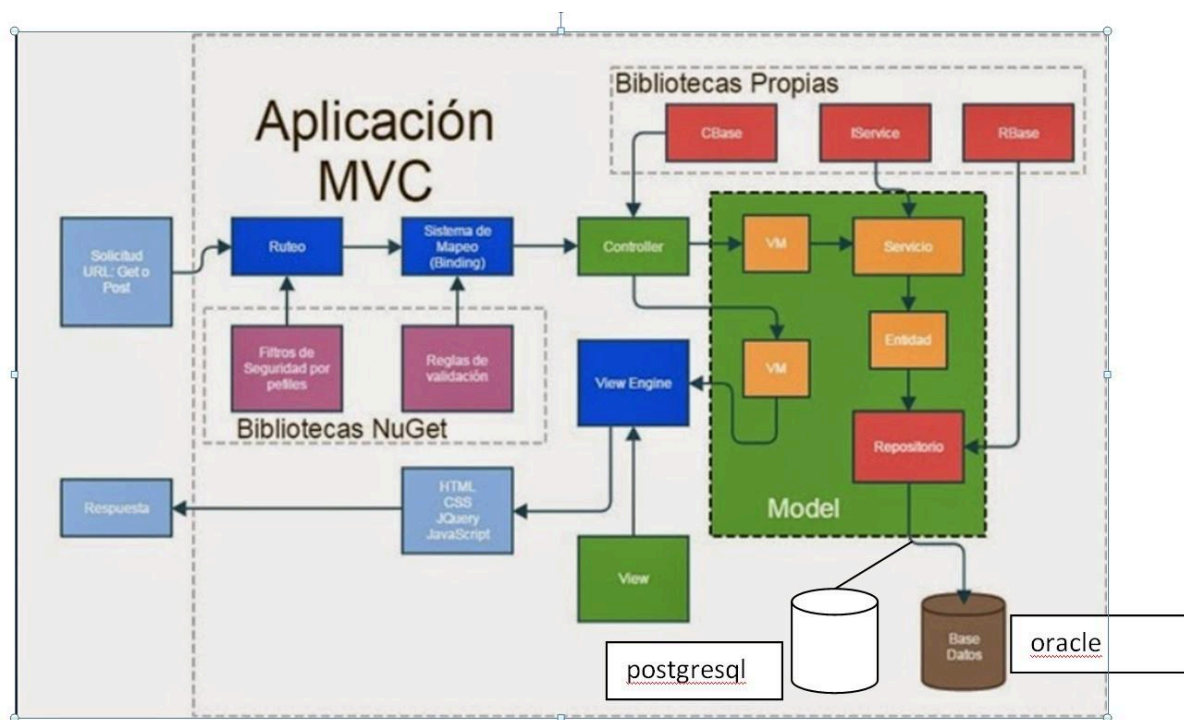


Diagrama de arquitectura FrontEnd



Arquitectura MVC en el backend con Spring Boot.



Componente Función principal

Modelo: Representa los datos y la lógica de negocio.

Vista: Es la interfaz de usuario. Muestra los datos al usuario.

Controlador: Recibe las entradas del usuario y las gestiona.

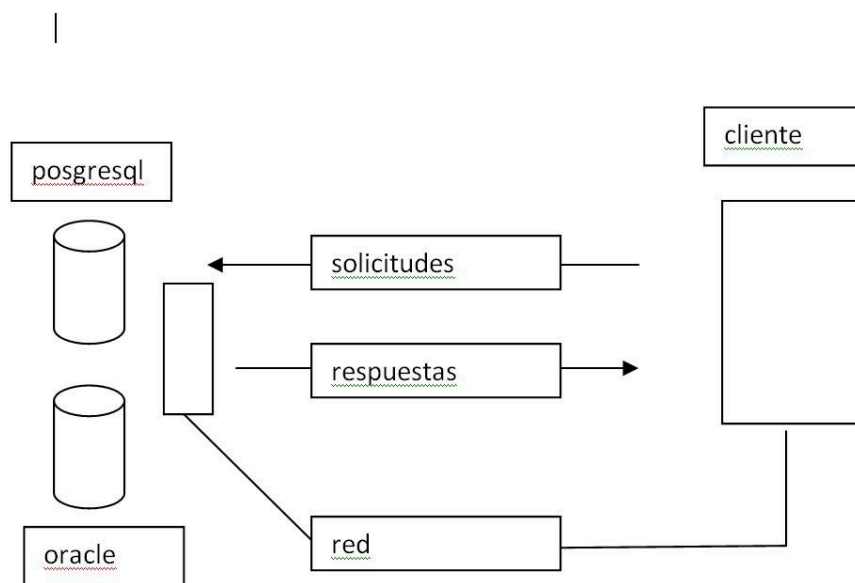
Ejemplo en Spring Boot (MVC Backend):

Modelo: Clases Usuario, Producto, etc. que representan entidades.

Vista: (en aplicaciones con UI en servidor como Thymeleaf). En REST APIs, no suele haber vista. normalmente podemos usar thymeleaf, o podemos usar workflow para el manejo de vista entre el modelo y el controlador

Controlador: Clases con @Controller o @RestController que manejan las peticiones.

Arquitectura Cliente-servidor.



Cliente: Solicita servicios (generalmente una aplicación web o móvil).

Servidor: Proporciona esos servicios (por ejemplo, procesamiento de datos, acceso a base de datos).

Características:

Comunicación generalmente a través de HTTP/HTTPS.

Separación clara entre frontend y backend.

Muy usada en aplicaciones web y móviles modernas.

Ejemplo con Angular y Spring Boot:

Ciente: Aplicación Angular que se ejecuta en el navegador.

Servidor: API REST construida con Spring Boot que procesa solicitudes y accede a la base de datos.

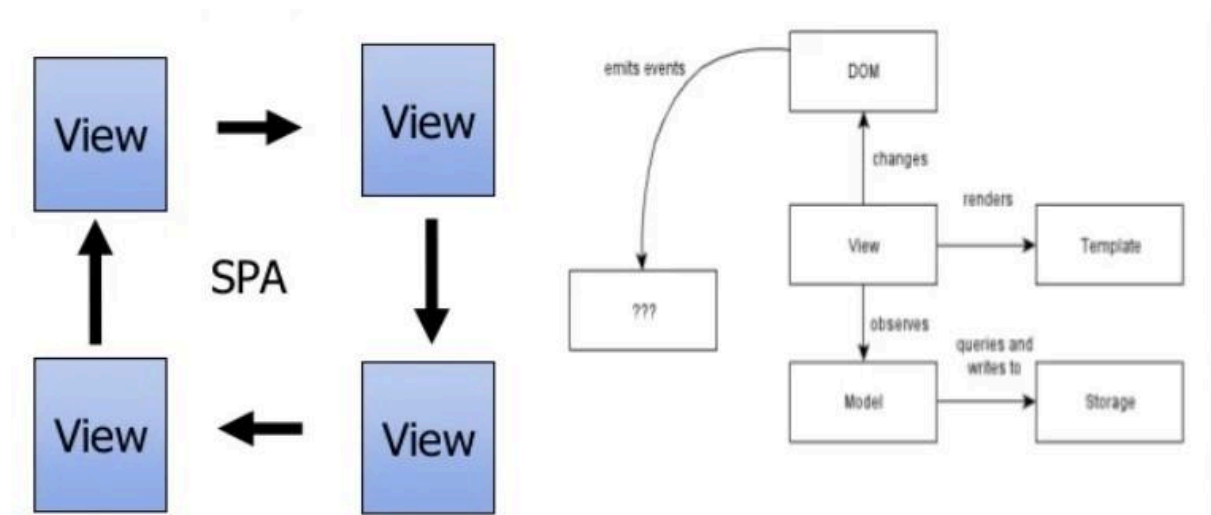
Ventajas:

Escalabilidad: puedes escalar el cliente y el servidor por separado.

Mantenimiento: separación de responsabilidades.

Reusabilidad del backend: distintos clientes (web, móvil) pueden usar la misma API.

Como funciona ... Sigle page application en Angular



Carga inicial única (index.html):

Angular carga una única página HTML (index.html) cuando entras a la app. Esta página contiene poco más que un `<app-root>` (el componente raíz).

Routing sin recarga de página:

Angular usa su propio enrutador (Router) para navegar entre diferentes "vistas" o componentes sin que el navegador recargue la página.

Ejemplo: navegar de inicio a productos no recarga la página; Angular simplemente cambia el componente que se muestra.

Renderizado dinámico con componentes:

El contenido se organiza en componentes. Cuando cambias de ruta, Angular muestra el componente correspondiente dentro del <router-outlet>.

API en segundo plano (Backend):

La aplicación consume datos a través de peticiones HTTP (como HttpClient) a APIs externas o servicios propios, y actualiza la vista dinámicamente.

Recursos estáticos cargados una vez:

Todos los recursos (HTML, JS, CSS) se cargan una vez, y luego Angular maneja la interacción del usuario sin nuevas cargas desde el servidor.

Explicación de Patrones de diseño Usados Repositorio, Service, Controller.

Back end

Patron Repository (Repositorio)

Propósito: Acceder a la base de datos.

Qué hace: Realiza operaciones CRUD (Create, Read, Update, Delete) sobre las entidades.

Tecnología en Spring: Se define como una interfaz que extiende JpaRepository o CrudRepository.

Patron Service (Servicio)

Propósito: Contiene la lógica de negocio.

Qué hace: Coordina y aplica reglas sobre los datos que vienen del repositorio antes de pasarlos al controlador.

Ventaja: Separa la lógica de negocio del controlador y del acceso a datos.

Patron Controller (Controlador)

Propósito: Maneja las peticiones HTTP.

Qué hace: Actúa como intermediario entre el cliente (Angular) y el backend (servicio).

Tecnología en Spring: Anotado con `@RestController`

Front end

Service (Servicio)

Propósito: Encapsula llamadas HTTP y lógica compartida.

Qué hace: Realiza peticiones al backend (Spring Boot), maneja la comunicación.

Tecnología: Usa HttpClient.

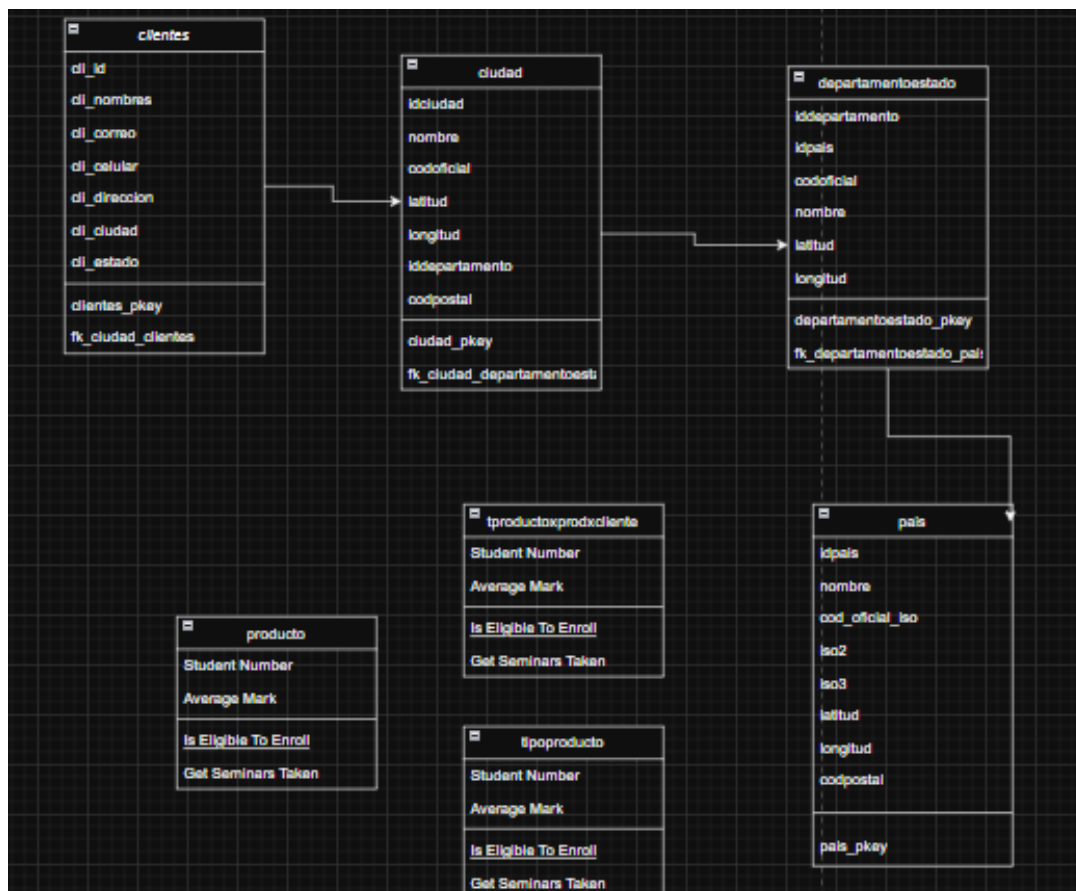
Component (Componente)

Propósito: Muestra la interfaz de usuario.

Qué hace: Llama a los servicios y muestra datos.

Tecnología: HTML + TypeScript (decorado con `@Component`).

Oracle:



Postgresql

Frontend (Angular):

- **Formulario con selects para (Bodega, Ciudad Destino, Ciudad Origen, Mercancia, Estado)**
- **Validaciones (Bodega obligatorio, origen y destino, cliente)**
- **Servicio envio.service.ts que llama a la API con HttpClient**
- **Vistas/envios que llama a un dialog nuevo-envio.component.ts**

Funcionalidad 2: Listar Envíos

Permite Listar todos los envío indicando su tipo (aéreo, marítimo o terrestre), origen, destino, peso, dimensiones, transportista y usuario responsable.

EndPoint

- **Frameworks: Spring Boot + Angular**
- **Base de datos: PostgreSQL y oracle**

Frontend (Angular):

- **Formulario Lista los principales detalles de los envíos Validaciones (Bodega**
- **Servicio envio.service.ts que llama a la API con HttpClient**
- **Vistas/envios que llama a el componente de listar envios envios.component.ts**

Fase de Pruebas

Pruebas Unitarias:

Las pruebas unitarias son un tipo de pruebas que se enfoca en verificar el correcto funcionamiento de la unidad más pequeña del código, normalmente se aplica sobre funciones o métodos de forma aislada. Las pruebas unitarias nos sirven para detectar errores en funciones específicas, verificar que el código haga lo que se espera.

JUnit para backend.

Se usó Junit con Mockito para probar el código del Backend, aquí les mostramos el código y las respuestas de las pruebas.

```

@Test
void testListarDetalleUsuarios() throws SQLException {

    List<Usuarios> usuariosMock = Arrays.asList(
        new Usuarios(1, "Juan", "calle falsa 123", "3164567212", "jgabriel@gmail.com",
            "41562258", "Admin123", 1, 1, 1),
        new Usuarios(1, "Maria", "calle 4 # 12 - 123", "3104567010", "maria@gmail.com",
            "170458962", "Admin1234-1", 1, 2, 1)
    );

    when(servicioUsuario.listadoUsuarios()).thenReturn(usuariosMock);

    MockHttpServletResponse response;
    try {
        response = mockMvc.perform(get("/tcc/gestion/CrudUsuarios/ListarUsuarios")
            .accept(MediaType.APPLICATION_JSON))
            .andReturn().getResponse();

        assertThat(response.getStatus()).isEqualTo(HttpStatus.OK.value());
    } catch (Exception e) {
        log.error(ExceptionUtil.format(e));
    }

}

84 @Test
85 void testBuscarUsuariosById() throws SQLException {
86
87
88     Usuarios usuariosMock =
89         new Usuarios(1, "Juan", "calle falsa 123", "3164567212", "jgabriel@gmail.com",
90             "41562258", "Admin123", 1, 1, 1);
91
92     when(servicioUsuario.buscarUsuariosXId(1)).thenReturn(usuariosMock);
93
94
95     MockHttpServletResponse response;
96     try {
97         response = mockMvc.perform(get("/tcc/gestion/CrudUsuarios/buscarUsuariosById")
98             .param("id", "1")
99             .accept(MediaType.APPLICATION_JSON))
100             .andReturn().getResponse();
101         verify(servicioUsuario).buscarUsuariosXId(1);
102
103
104     } catch (Exception e) {
105         log.error(ExceptionUtil.format(e));
106
107     }
108
109
110 }

```

Aquí está la ejecución de las pruebas:

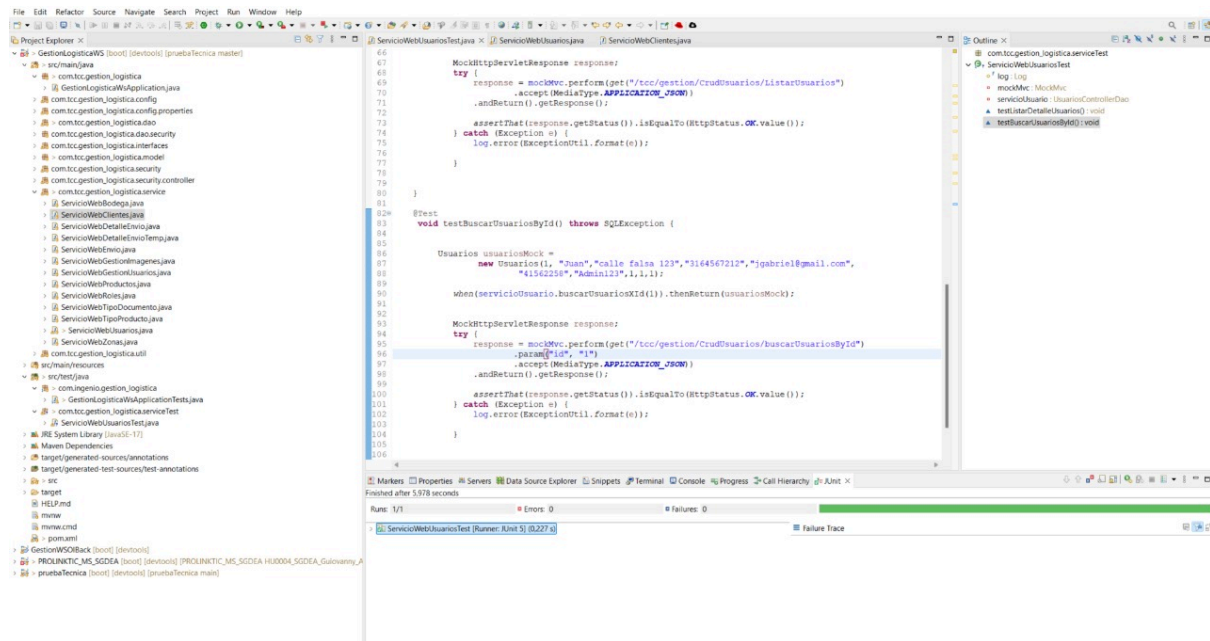
The image displays two screenshots of an IDE (IntelliJ IDEA) showing the execution of unit tests for a Java project. The project structure is visible on the left, and the test results are shown on the right.

First Screenshot (Successful Test):

- The test being executed is `testListaDetalleUsuarios` in the `ServiceWebUsuariosTest` class.
- The test is successful, with a status of `OK`.
- The test results show a `MockMvc` object and a `MockMvc.perform()` call.
- The test results show a `MockMvc.perform()` call.

Second Screenshot (Failed Test):

- The test being executed is `testBuscarUsuariosById` in the `ServiceWebUsuariosTest` class.
- The test is failed, with a status of `Failed`.
- The test results show a `MockMvc` object and a `MockMvc.perform()` call.
- The test results show a `MockMvc.perform()` call.
- The test results show a `MockMvc.perform()` call.



PROCESOS DE MEJORA

Agregar reglas personalizables según el tipo de cliente:

Objetivo: Adaptar la logística a las características particulares del cliente y su entorno.

Ejemplos:

- Si la zona de entrega de la mercancía es rural, entonces aumentar la fecha de entrega a dos días.
- Si el cliente paga por prioridad, entonces enviará la mercancía de ese cliente en las siguientes horas.

Ventajas:

- Aumento en la precisión de las promesas de entrega.
- Mejora en la satisfacción del cliente mediante un servicio personalizado.
- Adaptabilidad ante condiciones logísticas complejas.

Seguimiento en tiempo real de la mercancía:

Ejemplos:

- Que cuando la mercancía esté en camino, ya sea avión, camión o barco, el usuario pueda ver en el mapa la ruta de su mercancía.
- Que cuando se esté repartiendo la mercancía, el sistema organice las entregas dependiendo la dirección de cada usuario, de tal forma que se pueda disminuir los tiempos de entrega.

Ventajas:

- Mayor confianza y tranquilidad del cliente.
- Reducción en los tiempos de entrega mediante **ruteo inteligente**.
- Mejora en la eficiencia operativa de los transportistas.
-

Arquitectura basada en microservicios:

Objetivo: Facilitar el mantenimiento, escalabilidad y evolución del sistema.

Características:

- Cada módulo (seguimiento, reglas de entrega, optimización de rutas) es **independiente y desacoplado**.
- Se pueden actualizar o escalar funcionalidades **sin afectar el sistema completo**.

Ventajas:

- Mayor resiliencia ante errores.
- Posibilidad de escalar vertical u horizontalmente según la demanda.
- Desarrollo más ágil e independiente por cada equipo técnico.
-

Modo simulación:

Objetivo: Permitir pruebas seguras de rutas y decisiones logísticas antes de llevarlas a producción.

Funcionalidades:

- Simulación de nuevas rutas de entrega para **diversificación y reducción de riesgo**.
- Evaluación de escenarios alternativos en función de tiempo, tráfico, distancia y costos.

Ventajas:

- Anticipación a posibles cuellos de botella.
- Ahorro en costos por pruebas físicas innecesarias.
- Mejora continua basada en datos simulados realistas.

Conclusiones

La precisión en la geolocalización es crucial para garantizar que los vehículos de mensajería sigan las rutas correctas, evitando desvíos que puedan resultar en extravíos de mercancía o retrasos. La integración de tecnologías avanzadas como GPS de alta precisión y algoritmos de optimización en tiempo real es esencial para lograr una gestión eficiente y confiable de las rutas.

La optimización dinámica de rutas, basada en condiciones del tráfico y el clima, también es fundamental para garantizar entregas rápidas y seguras.

Los clientes finales son esenciales en el proceso, pero su poder sobre las decisiones operativas es limitado. Sin embargo, mantenerlos informados sobre el estado de sus entregas (por ejemplo, con notificaciones automáticas) aumentará la satisfacción y la confianza en el servicio.

Es importante no solo cumplir con sus expectativas en cuanto a tiempo y seguridad, sino también proporcionarles transparencia en el proceso de entrega. Esto ayuda a fortalecer la relación con el cliente y reduce la posibilidad de quejas o insatisfacción.

El desarrollo de un sistema de geolocalización para carros de mensajería es una solución innovadora que mejora la eficiencia, la precisión en las entregas y la satisfacción del cliente. Sin embargo, cada uno con sus propios intereses y niveles de influencia. Un enfoque equilibrado que considere tanto la optimización tecnológica como las necesidades y expectativas de las partes interesadas garantizará que el proyecto no solo sea funcional, sino también rentable y sostenible en el tiempo.

Ciclo de vida del desarrollo

1. Análisis de requisitos

- Se identificaron las necesidades específicas del sistema, como el manejo de inventarios, seguimiento de pedidos, control de transporte y generación de reportes.
- La comunicación con los usuarios clave (como operadores logísticos y administradores) fue fundamental para definir funcionalidades reales y evitar suposiciones erróneas.
- Se priorizaron requisitos funcionales y no funcionales (seguridad, rendimiento, escalabilidad).

2. Diseño del sistema

- Se estructuró el sistema en MVC el backend y el frontend , lo cual favorece la mantenibilidad.
- Se diseñaron los modelos de datos (UML, ERD) considerando relaciones entre entidades como clientes, transportistas, pedidos y almacenes.
- Se seleccionaron tecnologías adecuadas como Spring Boot (backend), Angular (frontend) y bases de datos relacionales postgresql o tan sensible y oracle para manejo de información sensible.

3. Implementación

- La separación de responsabilidades permitió una codificación más limpia y testeable.
- Se utilizaron buenas prácticas como control de versiones (Git), pruebas unitarias y principios SOLID.
- El desarrollo iterativo facilitó validar funcionalidades con entregas parciales.

4. Pruebas

- Se realizaron pruebas unitarias y de integración las cuales quedan pendientes por fases posteriores del proyecto con el fin de asegurar la lógica de negocio.
- Se identificaron errores en flujos críticos como el registro de envíos o la asignación de transportistas, que fueron corregidos antes de producción.
- Las pruebas con usuarios reales ayudaron a detectar errores de usabilidad y validación de datos.

5. Despliegue

- Se prepararon contenedores Docker y scripts de despliegue para asegurar consistencia entre ambientes.
- Se usa Openshift y rosa para el uso de CI/CD para realizar un mejor manejo de despliegue de la aplicación
- Se configuró un entorno de testing para validar el sistema antes del paso a producción.
- Se documentó el proceso de despliegue y los requisitos de infraestructura.

6. Mantenimiento

- Se corrigieron errores reportados tras la entrega inicial y se dejaron listos en el documento para corrección en fase posteriores.
- Se atendieron solicitudes de mejora basadas en la retroalimentación de usuarios.
- Se evaluó la necesidad de refactorizar partes del sistema con bajo rendimiento o alta complejidad.

Prototipo de baja fidelidad:

<https://balsamiq.cloud/sbpx3qs/panyh3k>

Prototipo de alta fidelidad:

<https://www.figma.com/design/gDEns4nNzo4RawTKrfoESI/TCCGestionLogistica-team-library?node-id=0-1&t=NaOoP28WQ26apMYV-1>

Url video explicativo

<https://youtu.be/6H1Zh3YhCt8>

url que contiene las fuentes del proyecto en su fase creación de prototipos iniciales

<https://github.com/guiovannycaro/gestionLogistica.git>

url directa al tablero de jira

<https://guiovannycaro.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog>

Referencias

<https://excelparatodos.com/matriz-de-riesgos/>

<https://blog.comparasoftware.com/matriz-de-riesgos-que-es/>

<https://www.piranirisk.com/es/blog/matriz-de-riesgos-que-es-ejemplos-y-como-crearla-facil>

<https://medium.com/@requeridosblog/requerimientos-funcionales-y-no-funcionales-ejemplos-y-tips-aa31cb59b22a>

<https://visuresolutions.com/es/gu%C3%ADa-de-trazabilidad-de-gesti%C3%B3n-de-requisitos/requisitos-funcionales-vs-no-funcionales/>

<https://www.lucidchart.com/blog/es/como-elaborar-diagramas-de-arquitectura>

<https://aws.amazon.com/es/what-is/architecture-diagramming/>

<https://www.yarquitectura.com/diagrama-de-arquitectura-de-software/>

<https://visuresolutions.com/es/gu%C3%ADa-de-trazabilidad-de-gesti%C3%B3n-de-requisitos/requisitos-funcionales-vs-no-funcionales/>

<https://single-spa.js.org/docs/ecosystem-angular/>

<https://es.single-spa.js.org/docs/examples/>

<https://refactoring.guru/es/design-patterns>

Pressman, Roger S., (2021) Ingeniería de software. McGraw-Hill Interamericana. Capítulo 24, 25, 26 página de la 490 a 548

Gual Ortí, J. (2016). Fundamentos del modelado y prototipado virtual en el diseño de productos.. D - Universitat Jaume I. Servei de Comunicació i Publicacions. Capítulo 2 y 3 páginas 3 a la 5.

Suárez, E. C. (2017). Prototipo, Contexto e Ingeniería del Software. Estudios de Postgrado, en Sistemas de Información, 2-20.

